ECE4063 IC Design Project

AES-128 Hardware Accelerator

Quartus, ModelSim, and Tiny Tapeout Workflow Report

Course	ECE4063 IC Design Project
Semester	S1-2026
Project Group ID	MSMC
Student 1	Chin Wei Chun, 33520569, wchi0051@student.monash.edu
Student 2	Foong Yao Le, 33521174, yfoo0021@student.monash.edu
Lecturer / Demonstrator	Dr. Patrick Ho
Submission Date	29/05/2026
Repository / Tiny Tapeout URL	Project: https://github.com/xcore11/aes128_project_G TinyTapeout for Optimized: https://github.com/xcore11/ttsky-aes128-optimized TinyTapeout for Baseline: https://github.com/xcore11/ttsky-aes128-baseline

Executive Summary

This project developed, verified, synthesized, and compared two AES-128 hardware accelerator implementations: a baseline design and an area-optimized design. Both designs implement AES-128 encryption for a 128-bit plaintext block and 128-bit key, including key expansion, control sequencing, datapath operations, and ciphertext output handling. The project scope was limited to AES-128 encryption only; AES decryption, AES-192, AES-256, side-channel protection, and full SoC integration were excluded to keep the work focused on standalone encryption hardware design and implementation comparison.

The baseline design was developed as a functionally correct reference architecture using a one-round-per-cycle iterative datapath with more parallel AES hardware. In contrast, the optimized design uses a shared-resource multi-cycle architecture that reuses one fixed-Boolean S-box and one MixColumns column unit across multiple cycles to reduce hardware area. This creates a clear comparison between a lower-latency parallel implementation and a smaller area-focused implementation.

Functional verification was performed using module-level tests and full-core self-checking ModelSim testbenches. Both implementations passed all 288 AES-128 encryption test vectors. Quartus synthesis confirmed that both designs compiled successfully and met the 50 MHz timing target. The baseline design completed encryption in 11 cycles, while the optimized design required 279 cycles. However, the optimized design reduced Quartus logic elements from 4,941 to 1,459, although register usage increased from 398 to 606 due to additional FSM control, counters, and temporary storage.

For Tiny Tapeout backend preparation, both designs were adapted using an 8-bit byte-loading wrapper to satisfy limited external I/O constraints. Backend results showed that the optimized design required only 4x2 tiles, while the baseline required 8x2 tiles. The optimized design also reduced wire length from 726,069 µm to 259,031 µm and combinational logic count from 5,294 to 1,444. Overall, the project demonstrates a clear area-latency trade-off: the optimized design sacrifices throughput and latency but provides a much smaller and more backend-suitable AES-128 implementation for area-constrained ASIC-style deployment.

Table of Contents

1.0 Introduction

This project focuses on the design, verification, synthesis, and backend preparation of an AES-128 hardware accelerator. AES-128 is a symmetric encryption algorithm that encrypts a 128-bit plaintext block using a 128-bit secret key to produce a 128-bit ciphertext. In this project, the AES-128 algorithm is implemented as synthesizable digital hardware using SystemVerilog, simulated using ModelSim, and synthesized using Quartus Prime.

The main objective of the project is not only to produce a functionally correct AES-128 encryption core, but also to demonstrate how a cryptographic algorithm can be organised as hardware. The design therefore includes the main AES datapath, key expansion logic, control sequencing, state storage, and output handling. The encryption process is controlled through a simple start/busy/done interface, where the input plaintext and key are provided to the design, the encryption operation is started, and the ciphertext output is read after completion.

The project also compares a baseline AES-128 implementation with an optimized implementation. The baseline design provides a functionally correct reference architecture, while the optimized design modifies the internal hardware organisation to reduce area. This allows the project to evaluate the trade-off between hardware resource usage, cycle count, timing, and implementation complexity.

The complete workflow includes RTL development, module-level and full-core simulation, verification using known AES-128 test vectors, Quartus compilation, synthesis result analysis, and Tiny Tapeout backend workflow preparation. The report presents the AES algorithm background, overall hardware architecture, design choices, RTL implementation, verification strategy, simulation evidence, Quartus synthesis results, backend evidence, and engineering trade-off analysis.

1.1 Project Requirements and Design Objectives

The main requirements and objectives of the project are summarised in Table 1.1.

Table 1.1: Project Requirements and Design Objectives

Requirement / objective	How the final project addresses it	Evidence location
AES-128 encryption only	The design accepts a 128-bit plaintext and 128-bit key, then generates a 128-bit ciphertext. Decryption, AES-192, and AES-256 are outside the project scope.	Sections 2, 5, 6
Hardware implementation using RTL	The AES-128 encryption core is implemented using synthesizable SystemVerilog modules.	Sections 3, 5, Appendix A
Key expansion, control, datapath, and output handling	The architecture includes a key expansion block, control FSM, round counter, AES round datapath, state register, and ciphertext output register.	Sections 3 and 5
Functional verification	The design is verified using module-level tests and full AES-128 encryption testbenches. Known AES-128 test vectors are used to check correctness.	Section 6, Appendix B
Baseline and optimized comparison	A baseline design and an optimized design are implemented and compared in terms of resource usage, timing, cycle count, latency, and throughput.	Sections 4, 7, and 9
Quartus compilation and synthesis evidence	Both designs are compiled in Quartus and evaluated using resource utilisation and timing reports.	Section 7
Tiny Tapeout backend workflow	A Tiny Tapeout-compatible version of the design is prepared for backend hardening and evidence collection.	Section 8
Engineering trade-off analysis	The report analyses the impact of architecture and resource-sharing decisions on area, latency, timing, and complexity.	Sections 9 and 10
Documentation and citation declaration	The final report includes architecture explanation, verification evidence, synthesis results, citations, and contribution statement.	Sections 11 and 12

1.2 Project Scope and Exclusions

The scope of this project is limited to AES-128 encryption hardware implementation. The design processes one 128-bit plaintext block with one 128-bit cipher key and produces one 128-bit ciphertext block. The implemented hardware includes the AES encryption datapath, key expansion logic, control sequencing, state storage, and output handling. The design is verified using simulation and synthesized using Quartus Prime.

The following items are included in the project scope:

- AES-128 encryption core
- 128-bit plaintext input
- 128-bit key input
- 128-bit ciphertext output
- Key expansion for AES-128 round keys
- Control logic for encryption sequencing
- Baseline and optimized hardware implementations
- ModelSim simulation and waveform evidence
- Quartus synthesis and timing/resource analysis
- Tiny Tapeout backend workflow evidence

The following items are excluded from the project scope:

- AES decryption
- AES-192 and AES-256
- Side-channel countermeasures
- Processor subsystem
- DMA engine
- Large SoC wrapper
- Full local ASIC backend installation

These exclusions keep the project focused on the design and evaluation of a standalone AES-128 encryption hardware core.

1.3 Report Structure

This report is organised according to the complete AES-128 hardware design workflow, from algorithm background to RTL implementation, verification, synthesis, backend preparation, and engineering analysis.

Table 1.2: Report Structure

Section	Title	Main Content
2	AES-128 Algorithm Background	Introduces AES-128 encryption, including the round structure, state transformation operations, and key expansion process.
3	Overall Hardware Architecture	Presents the high-level AES hardware structure, including the top wrapper, control FSM, key expansion block, round datapath, state register, and output handling.
4	Engineering Design Choices	Explains the selected AES architecture style, wrapper interface, S-box design, and optimization target. It also defines the baseline and optimized designs.
5	RTL Architecture and Implementation	Describes the SystemVerilog module hierarchy, top-level interface, control FSM, datapath operation, byte ordering, S-box, MixColumns, and key expansion implementation.
6	Functional Verification	Explains the verification methodology, testbench structure, AES-128 test vectors, simulation logs, waveform evidence, and demonstration readiness.
7	Quartus Compilation and Synthesis Evidence	Presents Quartus project configuration, resource utilisation, timing results, compilation status, and warning or constraint discussion.
8	Tiny Tapeout Backend Workflow Evidence	Documents the Tiny Tapeout preparation, wrapper adaptation, hardening workflow result, generated layout evidence, and backend discussion.
9	Performance Comparison and Optimization Analysis	Compares the baseline and optimized designs using cycle count, latency, throughput, Fmax, timing slack, and resource utilisation.
10	Engineering Discussion, Limitations, and Future Improvements	Discusses the main area-latency trade-offs, design limitations, and possible future extensions.
11	Submission Package Organisation	Summarises the final submission folder structure and key files included in each folder.
12	Academic Integrity, Citations, and Contribution Statement	Declares external sources, reused materials, citations, and individual contribution responsibilities.
13	Conclusion	Summarises the project outcome, verification result, synthesis result, backend completion, and key engineering lesson.

This structure ensures that the report follows the full development path of the AES-128 hardware accelerator and clearly links design decisions to verification, synthesis, backend evidence, and engineering trade-off analysis.

2.0 AES-128 Algorithm Background

The Advanced Encryption Standard (AES) is a symmetric block cipher standardized by the National Institute of Standards and Technology (NIST). AES operates on fixed 128-bit data blocks and supports 128-bit, 192-bit, and 256-bit key lengths [1]. This project implements only the AES-128 encryption mode, where both the plaintext block and secret key are 128 bits. AES-128 performs 10 encryption rounds and generates a 128-bit ciphertext output from the input plaintext and key. Decryption, AES-192, AES-256, and side-channel countermeasures are outside the scope of this project.

Internally, AES represents the 128-bit data block as a 4 × 4 byte state matrix. The bytes are arranged column by column, and each encryption round transforms this state using a sequence of substitution, permutation, mixing, and key addition operations. For AES-128, the original 128-bit key is expanded into 11 round keys: one initial round key and ten additional round keys for the ten encryption rounds. The implemented hardware therefore requires both a datapath for state transformation and a key expansion block to generate the correct round key for each stage.

In hardware implementation, AES-128 is realised as a digital datapath controlled by sequential logic. The datapath performs the AES round operations on a 128-bit internal state, while the key expansion block generates the required round keys from the original cipher key. A control unit is used to sequence the initial AddRoundKey step, the nine main rounds, and the final round. Different hardware architectures may implement these operations using different resource-sharing approaches, which creates trade-offs between area, timing, latency, and design complexity.

2.1 AES-128 Round Structure

AES-128 encryption consists of an initial AddRoundKey step, nine main rounds, and one final round. The first step combines the plaintext with the original secret key. Rounds 1 to 9 then apply four operations: SubBytes, ShiftRows, MixColumns, and AddRoundKey. The final round applies SubBytes, ShiftRows, and AddRoundKey only, with MixColumns omitted. This round structure is defined by the AES standard and forms the algorithmic basis for the hardware implementation in this project.

Table 2.1: AES-128 Encryption Round Structure

AES Stage	Operations Included	Hardware Implementation Note
Initial step	• AddRoundKey	The plaintext is XORed with RoundKey0, which is the original 128-bit key.
Rounds 1-9	• SubBytes • ShiftRows • MixColumns • AddRoundKey	The main AES round datapath transforms the state and combines it with the current round key.
Round 10	• SubBytes • ShiftRows • AddRoundKey	The controller skips MixColumns in the final round before producing the ciphertext.

The AddRoundKey operation XORs the current state with the round key. This operation directly combines the current datapath state with the secret key material. Since XOR is simple in hardware, AddRoundKey is implemented as a combinational 128-bit XOR operation.

The SubBytes operation replaces each byte of the state using the AES substitution box, or S-box. This introduces non-linearity into the cipher, making the relationship between plaintext, key, and ciphertext harder to predict. In hardware, this operation can be implemented using lookup tables, case statements, ROM-style structures, or Boolean logic.

The ShiftRows operation rearranges bytes within the AES state matrix. The first row is unchanged, while the second, third, and fourth rows are cyclically shifted by different offsets. This spreads byte positions across the state and prepares the data for column mixing.

The MixColumns operation mixes the four bytes in each column using finite-field arithmetic. Its purpose is to provide diffusion, meaning that a change in one input byte affects multiple output bytes. In hardware, this is mainly implemented using XOR operations and multiplication-by-two logic, commonly called xtime.

The overall AES-128 encryption sequence is summarised in Figure 2.1.

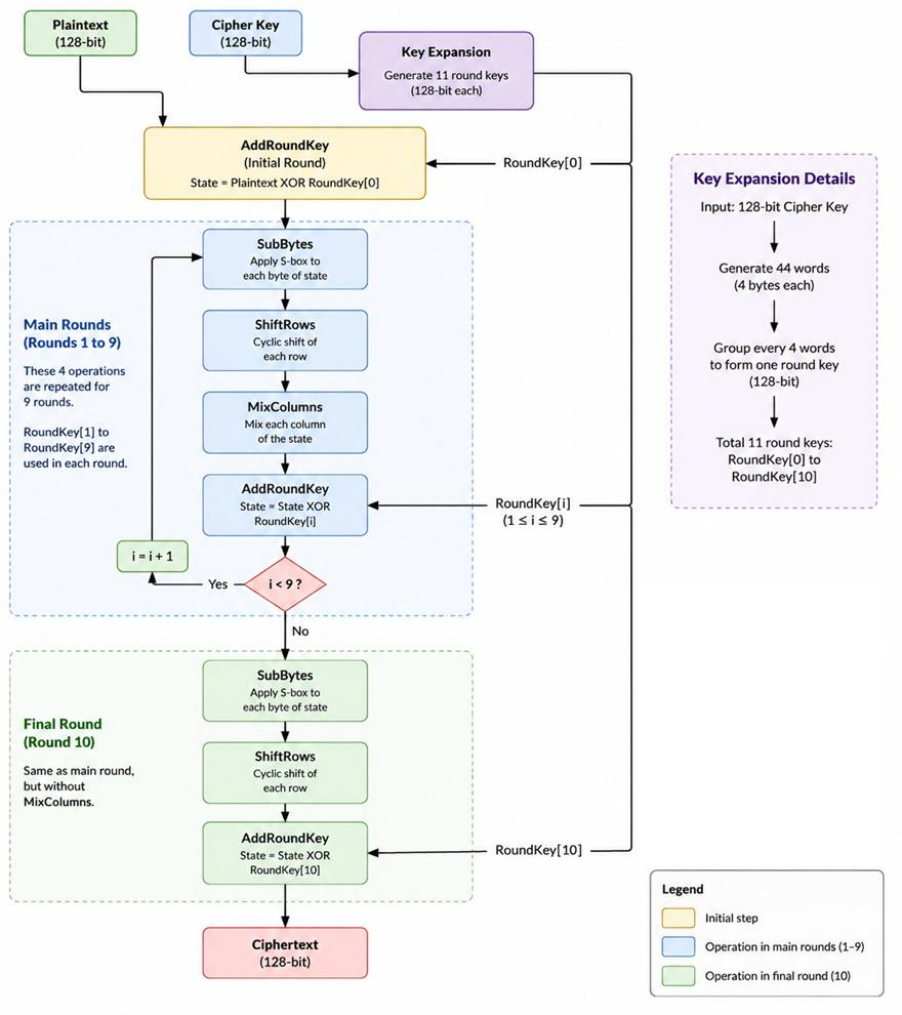

Figure 2.1: AES-128 Encryption Flow

2.2 Key Expansion Summary

AES-128 does not use the original 128-bit cipher key directly for every encryption round. Instead, the cipher key is expanded into a set of round keys, where each round key is used by the AddRoundKey operation at a different stage of encryption. For AES-128, the original key produces 11 round keys in total:

- RoundKey[0] for the initial AddRoundKey step
- RoundKey[1] to RoundKey[10] for the ten encryption rounds.

The 128-bit cipher key is first divided into four 32-bit words, usually labelled as W0, W1, W2, and W3. These four words form RoundKey[0]. The key expansion process then generates the remaining words until a total of 44 words are produced. Every group of four words forms one 128-bit round key.

For each new round-key group, the first word is generated using the previous word after three transformations: RotWord, SubWord, and Rcon. RotWord cyclically rotates the previous 32-bit word by one byte. SubWord applies the AES S-box substitution to each byte of the rotated word. Rcon provides a round-dependent constant that is XORed into the transformed word. The result is XORed with the word four positions earlier to produce the first word of the new round key.

The remaining three words in the same round-key group are generated using XOR chaining only. Each new word is produced by XORing the word four positions earlier with the immediately previous generated word. Therefore, for a new round key containing W4, W5, W6, and W7, the generation process can be written as:

- W4 = W0 XOR SubWord(RotWord(W3)) XOR Rcon[1]
- W5 = W1 XOR W4
- W6 = W2 XOR W5
- W7 = W3 XOR W6

This process repeats until RoundKey[10] is generated. The AES-128 key expansion process is summarised in Figure 2.2.

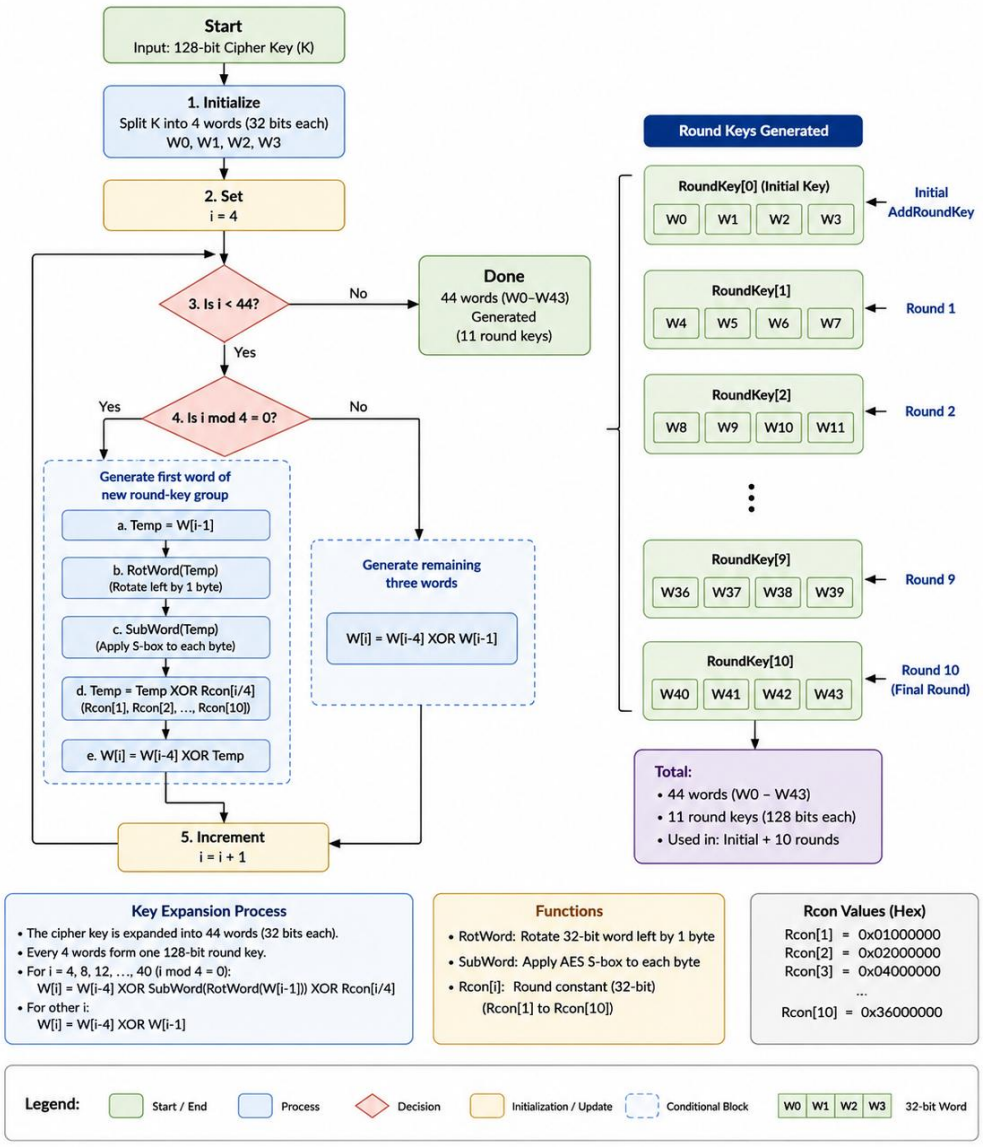

Figure 2.2: AES-128 Key Expansion Flow

3.0 Overall Hardware Architecture

The implemented AES-128 hardware accelerator follows a modular architecture consisting of a top-level wrapper, control logic, key expansion logic, AES round datapath, state registers, and output handling. This structure matches the project requirement for an AES-128 encryption core that includes key expansion, a control FSM, datapath, and ciphertext output handling.

At the top level, the AES core receives a 128-bit plaintext block, a 128-bit cipher key, a clock, a reset signal, and a start control signal. Once encryption begins, the control logic sequences the AES operation from the initial AddRoundKey step through the main encryption rounds and the final round. The datapath updates the internal 128-bit AES state, while the key expansion block provides the required round key for each stage. After the final round is completed, the 128-bit ciphertext is stored at the output and the done signal is asserted.

The overall architecture is organised so that the control logic and datapath operate together during encryption. The control logic determines when each AES stage is executed, while the datapath performs the 128-bit state transformation. The key expansion block supplies the correct round key for the initial step, the nine main rounds, and the final round. This modular structure makes the design easier to verify, debug, synthesize, and explain, while satisfying the project requirement for an AES-128 core with key expansion, control, datapath, and output handling.

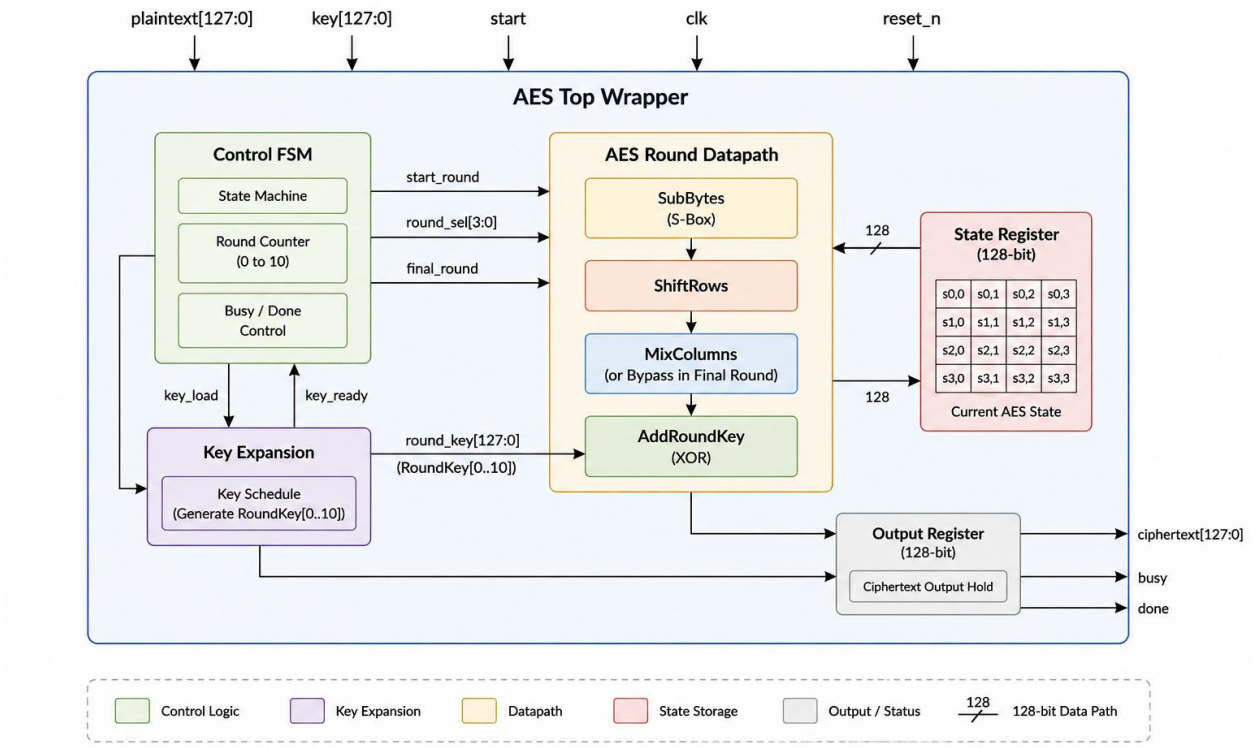

Figure 3.1: Overall AES-128 Hardware Architecture

As shown in Figure 3.1, the AES top wrapper connects the external control and data signals to the internal control FSM, key expansion block, state storage, round datapath, and output register.

3.1 Architecture Summary

The main functional blocks of the AES-128 hardware architecture are summarised in Table 3.1.

Table 3.1: AES-128 Hardware Architecture Summary

Block	Purpose	Important Design Detail to Explain
AES Top Wrapper	Provides the main external interface for the AES-128 encryption core.	The AES top wrapper connects the external input and output ports to the internal AES core. It receives the 128-bit plaintext block, 128-bit cipher key, clock, reset, and start signal. It also outputs the 128-bit ciphertext, busy status, and done status. The wrapper is responsible for exposing a clean and simple interface to the testbench or external system, while hiding the internal datapath, key expansion, and control details. It ensures that the AES core can be started using a single control signal and that the encrypted result can be read once the operation is complete.
Control FSM	Controls the sequencing of the AES encryption process.	The control FSM manages the overall encryption flow. It determines when the design is idle, when a new encryption operation starts, when the initial AddRoundKey step is performed, when the main AES rounds are executed, and when the final round is completed. It also controls the busy and done signals so that the external interface can identify whether the core is processing or finished. The FSM ensures that the AES operations occur in the correct order and prevents the ciphertext output from being updated before the full encryption sequence is complete.
Round Counter	Tracks the current AES round during encryption.	The round counter is used by the control logic to identify which stage of AES-128 encryption is currently being executed. Since AES-128 uses 10 encryption rounds after the initial AddRoundKey step, the counter is used to separate the main rounds from the final round. During rounds 1 to 9, MixColumns is included in the datapath. During round 10, the counter allows the controller to bypass MixColumns because the final AES round only performs SubBytes, ShiftRows, and AddRoundKey. The round counter is also used to select or generate the correct round key for the current stage.
State Register	Stores the current 128-bit AES internal state.	The state register holds the intermediate 128-bit data block as it passes through the AES encryption process. At the start of encryption, the state is initialised using the result of the plaintext XORed with RoundKey[0]. After each round, the output of the AES round datapath is written back into the state register. This allows the same internal state to be updated progressively from the initial round through to the final round. Correct byte ordering is important because AES treats the 128-bit block as a 4 × 4 byte matrix arranged column by column.
Key Expansion	Generates the required round key for each AES stage.	The key expansion block takes the original 128-bit cipher key and derives the round keys used throughout AES-128 encryption. AES-128 requires 11 round keys in total: RoundKey[0] for the initial AddRoundKey step and RoundKey[1] to RoundKey[10] for the ten encryption rounds. The key expansion process uses word-based operations such as RotWord, SubWord, Rcon, and XOR chaining to generate the required round keys. These round keys are then provided to the AddRoundKey stage of the datapath at the correct encryption round.
Round Datapath	Performs the AES transformation operations on the state register value.	The AES round datapath is the main computational part of the encryption core. It transforms the current 128-bit state using the AES operations. During the main rounds, the datapath performs SubBytes, ShiftRows, MixColumns, and AddRoundKey. SubBytes applies S-box substitution to each byte, ShiftRows rearranges bytes across the state matrix, MixColumns mixes each column using finite-field arithmetic, and AddRoundKey XORs the result with the current round key. During the final round, MixColumns is bypassed so that the datapath performs only SubBytes, ShiftRows, and AddRoundKey.
Output Register	Stores the final ciphertext and indicates completion.	The output register captures the final 128-bit AES state after the last AddRoundKey operation in round 10. Once this value is stored, it is presented as the ciphertext output. The done signal is asserted to indicate that the ciphertext is valid, while the busy signal is deasserted to show that the core has completed the encryption operation. Holding the ciphertext in an output register ensures that the result remains stable until the next encryption operation begins.

As summarised in Table 3.1, the AES-128 hardware architecture separates the design into clear functional blocks. The control FSM and round counter manage the sequencing of the encryption process, while the datapath blocks perform the required AES transformations on the 128-bit state. The key expansion block supplies the round keys required by AddRoundKey, and the output register stores the final ciphertext once encryption is complete. This modular organisation improves readability and verification because each functional block can be understood and tested separately before full-system integration.

3.2 Data and Control Flow

The data and control flow of the AES-128 hardware accelerator is summarised in Table 3.2.

Table 3.2: AES-128 Data and Control Flow

Step	Operation	Expected Signal Behaviour
1	Reset / idle	When reset is asserted, the AES core returns to a known idle condition. Internal registers such as the state register, round counter, internal working registers, ciphertext register, busy, and done are cleared or reset to their default values. In the idle state, busy = 0 and done = 0, indicating that the core is ready to accept a new encryption command.
2	Start accepted	When start signal is asserted while the core is idle, the encryption process begins. The 128-bit plaintext and 128-bit key are accepted by the AES core and used to initialise the encryption sequence. The control logic then asserts busy to indicate that encryption is in progress. The done signal remains low because the ciphertext is not valid yet.
3	Initial AddRoundKey	The first AES operation is performed by XORing the plaintext with RoundKey[0], which is the original cipher key. The result becomes the initial 128-bit internal AES state. This prepares the state for the main AES encryption rounds.
4	Rounds 1-9	The round counter increments through rounds 1 to 9. In each main round, the state is transformed using SubBytes, ShiftRows, MixColumns, and AddRoundKey. The key expansion block provides the correct round key for the current round. After each round, the updated 128-bit result is stored back into the state register.
5	Final round	During the final round, the controller bypasses MixColumns. The datapath performs only SubBytes, ShiftRows, and AddRoundKey using RoundKey[10]. This follows the AES-128 encryption structure, where MixColumns is omitted in the final round.
6	Completion	After the final AddRoundKey operation, the final 128-bit state is stored as the ciphertext output. The ciphertext signal becomes stable and valid. The done signal is asserted to indicate completion, while busy is deasserted to show that the AES core is no longer processing and can accept another encryption request.

As shown in Table 3.2, the AES core operates through a clear start-to-completion sequence. The control logic first accepts the input command, then sequences the initial AddRoundKey step, the nine main AES rounds, and the final round. The busy signal indicates active processing, while the done signal indicates that the ciphertext output is valid. This signal behaviour provides a simple and reliable interface for simulation, testbench verification, and demonstration.

4.0 Engineering Design Choices

This section summarises the main engineering design choices made for the AES-128 hardware accelerator. The project requires the design to define its AES-128 implementation style, wrapper interface, S-box implementation method, and optimization target. These choices are important because they directly affect the hardware area, timing behaviour, cycle count, verification complexity, and suitability for synthesis.

Two AES-128 implementations were developed for comparison. The baseline design was used as the reference implementation, while the optimized design was developed to demonstrate an area-focused improvement. Both designs perform AES-128 encryption only and are verified using the same AES-128 test vector set. The detailed RTL module hierarchy is discussed later in Section 5, while the detailed synthesis and performance comparison are discussed in Sections 7 and 9. This section focuses only on the design decisions and their engineering justification.

Table 4.1: Summary of Engineering Design Choices

Criterion	Final choice	Justification	Evidence Used
AES-128 style	**Baseline**: one-round-per-cycle iterative AES **Optimized**: low-area shared-resource multi-cycle datapath	The baseline design uses a one-round-per-cycle iterative AES architecture because it provides a clear and correct reference design. In this structure, one AES round is completed every clock cycle using parallel SubBytes, ShiftRows, MixColumns, and AddRoundKey logic. This makes the baseline easier to verify, but it requires more hardware because many AES operations are performed in parallel. The optimized design was changed to a low-area shared-resource multi-cycle datapath. This means that hardware blocks such as the S-box and MixColumns column unit are reused over multiple cycles instead of being duplicated. This choice requires a larger internal FSM to sequence byte-by-byte SubBytes, key expansion, column-by-column MixColumns, AddRoundKey, and round progression. The design therefore trades higher cycle latency for lower hardware area, which is suitable for area-constrained hardware implementation and relevant to Tiny Tapeout-style backend constraints.	Lower area Tiny Tapeout GDS tile render in optimized AES-128 (*Section 8.0*) Optimized AES-128 has Lower logic elements in Quartus compilation report (*Section 7.0*) Higher cycle count in optimized AES-128 from simulation results (*Section 6.0*)
Wrapper interface	Start / busy / done interface with 128-bit plaintext, 128-bit key, and 128-bit ciphertext ports.	The start/busy/done interface was selected because it gives a simple and deterministic way to control the AES accelerator. The testbench can apply the plaintext and key, pulse start, wait while busy is high, and check the ciphertext when done is asserted. This interface does use a small FSM, typically equivalent to an IDLE/RUNNING/DONE control flow, but this FSM mainly supports external handshaking rather than AES computation. Compared with a memory-mapped interface, it avoids address decoding, bus registers, read/write control, and extra status logic. Compared with a streaming interface, it avoids valid/ready buffering and continuous block-transfer control. Therefore, it is not the main area optimization, but it is area-friendly and keeps verification simple.	Wrapper Architecture implemented in *Section 5.1* Test benches utilizes simple flags to control AES accelerator and obtain results for both baseline and optimized (*Section 6.0*)
S-box design	**Baseline**: lookup-table S-box. **Optimized**: fixed Boolean logic S-box.	The baseline design uses a lookup-table S-box because it is easy to understand, easy to verify against the AES standard, and suitable for producing the first correct implementation. However, the baseline architecture duplicates the S-box many times because SubBytes requires 16-byte substitutions and key expansion requires additional SubWord substitutions. Since the S-box is one of the largest repeated blocks in AES, duplicating it has a major area cost. The optimized design therefore uses one shared S-box, which is reused for both the AES SubBytes operation and the key expansion SubWord operation. This choice directly contributes to area reduction, but it requires FSM-controlled byte selection, S-box input multiplexing, and output storage. The trade-off is that SubBytes and key expansion take more cycles, but the amount of duplicated S-box hardware is greatly reduced.	Higher cycle count in optimized AES-128 due to repeated module usage (*Section 6.0*) Lower logic elements in optimized AES-128 due to the utilization of single block in repeated cycles (*Section 7.0*) Lower area Tiny Tapeout GDS tile render in optimized AES-128 (*Section 8.0*)
Design optimization	**Optimized**: Reduce chip area through resource sharing.	The baseline was intentionally kept as a straightforward reference implementation rather than an aggressively optimized design. This made the AES round sequence, key expansion, and ciphertext output easier to simulate, debug, and verify before optimization. Keeping the baseline simple also provided a fair comparison point, so that any later changes in area, timing, and cycle count could be clearly attributed to the optimized shared-resource architecture. The optimized design then targeted area reduction by reusing one S-box and one MixColumns column unit across multiple cycles under FSM control. This reduced duplicated datapath hardware but increased encryption latency.	Lower logic elements in optimized AES-128 due to the utilization of single block in repeated cycles (*Section 7.0*) Lower area Tiny Tapeout GDS tile render in optimized AES-128 (*Section 8.0*)

4.1 Baseline and Optimized Design Definition

The two AES-128 implementations used in this project are defined in Table 4.2. The purpose of this comparison is not to change the AES algorithm, but to evaluate how different hardware organisation choices affect implementation results.

Table 4.2: Baseline and Optimized Design Definition

Version	Purpose	Main Implementation Features
Baseline design	The baseline design was developed as the first correct AES-128 hardware implementation. Its main purpose was to act as a functional reference before applying any major optimization.	The baseline uses a straightforward one-round-per-cycle iterative AES architecture. After start is asserted, the design performs the initial AddRoundKey operation, followed by the required AES rounds. Each round is implemented using parallel round hardware, including SubBytes, ShiftRows, MixColumns, AddRoundKey, and key expansion logic. The design uses multiple S-box instances so that byte substitution can be performed in parallel. This makes the baseline easier to understand, simulate, and debug, but it increases hardware area.
Optimized design	The optimized design was developed to reduce hardware area while maintaining the same AES-128 functional behaviour as the baseline. It is used to demonstrate the effect of architectural optimization.	The optimized design uses a low-area shared-resource multi-cycle AES datapath. Instead of duplicating expensive AES hardware, it reuses one S-box and one MixColumns column unit across multiple clock cycles. A larger internal FSM controls the sequence of key expansion, SubBytes, ShiftRows, MixColumns, AddRoundKey, round counting, and output completion. This reduces duplicated combinational logic, especially S-box and MixColumns hardware, but increases the number of cycles required for one encryption.

The baseline and optimized designs are compared using the same verification approach to ensure that both produce the correct AES-128 ciphertext for the same plaintext and key inputs. This makes the synthesis and performance comparison fair, because differences in area, timing, and latency are caused by hardware implementation choices rather than differences in algorithmic behaviour.

The engineering focus of this project is therefore the trade-off between parallel hardware execution and resource sharing. The baseline design favours fewer cycles by using more hardware resources, while the optimized design favours lower area by reusing selected resources over more cycles. This design comparison provides the basis for the later verification, Quartus synthesis, and performance analysis sections.

5.0 RTL Architecture and Implementation

The AES-128 hardware accelerator was implemented in SystemVerilog using a modular RTL structure. The design is separated into top-level wrapper logic, AES core control, key expansion, AES round datapath, S-box substitution, ShiftRows, MixColumns, AddRoundKey, and output handling. This modular approach improves readability and supports separate verification of individual AES operations before full-core integration.

Two RTL implementations were developed: a baseline implementation and an optimized implementation. Both designs use the same AES-128 encryption algorithm and the same external control concept, but their internal resource organisation differs. The baseline design uses a more parallel datapath to complete one AES round per cycle, while the optimized design reuses selected hardware resources across multiple cycles to reduce logic area. The following subsections describe the RTL interface, module hierarchy, control sequencing, datapath operation, and key implementation blocks.

5.1 Top-Level Interface

Both the baseline and optimized AES-128 implementations use the same top-level interface. This allows both designs to be verified using the same testbench structure and compared under the same input conditions. The interface follows a simple start / busy / done control style, where the external testbench provides a plaintext and key, asserts start, waits for done, and then checks the resulting ciphertext.

Table 5.1: AES-128 Top-Level Interface

Signal	Direction	Width	Purpose / Timing Behaviour
clk	Input	1 bit	Clock signal for the AES controller, datapath registers, key registers, and output register.
reset_n	Input	1 bit	Active-low asynchronous reset. When reset_n = 0, the AES core clears its internal state, control registers, status signals, and ciphertext output.
start	Input	1 bit	One-clock start request. Encryption is accepted only when the core is idle. If start is asserted while busy = 1, the request is ignored.
plaintext	Input	128 bits	Input plaintext block to be encrypted. It is accepted by the AES core when a valid encryption operation begins.
key	Input	128 bits	Input AES-128 secret key. It is used as the initial round key, RoundKey[0], and is expanded to generate the remaining round keys.
busy	Output	1 bit	High while encryption is running. In the baseline design, it remains high during the shorter round-processing sequence. In the optimized design, it remains high throughout the longer shared-resource FSM sequence.
done	Output	1 bit	One-clock completion pulse asserted when the ciphertext is valid. It is cleared by default on the following clock cycle.
ciphertext	Output	128 bits	Final AES ciphertext after round 10 is completed. It remains stored until the next reset or until a new encryption operation updates it.

The top-level interface is intentionally kept simple because the AES core processes one 128-bit block per encryption request. This makes the design easy to control in simulation and suitable for verification using known AES-128 test vectors. The testbench only needs to apply plaintext and key, assert start for one clock cycle, wait for done, and then compare ciphertext with the expected output. The top-level interface is shown as a black-box port diagram in Figure 5.1.

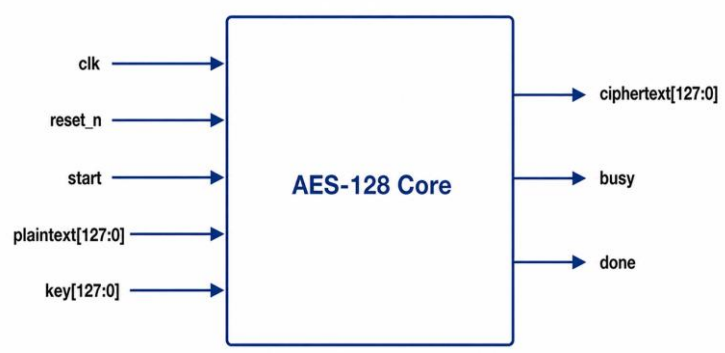

Figure 5.1: AES-128 Top-Level Interface Diagram

5.2 Module Hierarchy

The baseline and optimized AES-128 designs share the same external wrapper style but use different internal module hierarchies. Both designs expose the same top-level interface, allowing the same verification approach to be used for functional testing. However, the internal RTL organisation differs because the baseline design uses a more parallel AES datapath, while the optimized design uses shared hardware controlled by a more detailed FSM.

The baseline hierarchy separates the AES operations into dedicated modules such as SubBytes, ShiftRows, MixColumns, AddRoundKey, and KeyExpansion. This makes the design easier to trace and supports one-round-per-cycle execution. In contrast, the optimized hierarchy reduces the number of duplicated hardware resources by integrating more sequencing inside the AES core and reusing selected modules across multiple cycles.

5.2.1 Baseline Module Hierarchy

Table 5.2: Baseline RTL Module Hierarchy

Module / File	Role in Design	Key inputs	Key outputs	Notes
aes128_baseline.sv	Top-level wrapper for the baseline design.	• clk • reset_n • start • key • plaintext	• ciphertext • busy • done	Provides the external interface and instantiates the baseline aes_core.
aes_core.sv	Baseline AES controller and state register	• clk • reset_n • start • key • plaintext	• ciphertext • busy • done	Performs the initial AddRoundKey step, controls the round counter, updates the AES state, and completes one AES round per clock cycle.
aes_round.sv	One complete AES round datapath.	• state_in • round_key • final_round	• state_out	Performs SubBytes, ShiftRows, MixColumns, and AddRoundKey. MixColumns is bypassed when final_round is asserted.
sub_bytes.sv	Parallel SubBytes operation.	• state_in	• state_out	Instantiates 16 parallel S-boxes so that all state bytes are substituted in one round operation.
sbox.sv	AES S-box substitution.	• in_byte	• out_byte	Lookup-table style S-box used for byte substitution.
shift_rows.sv	AES ShiftRows byte permutation.	• state_in	• state_out	Pure combinational byte rewiring based on the AES state matrix arrangement.
mix_columns.sv	Full-state MixColumns operation.	• state_in	• state_out	Processes all four AES columns in parallel using MixColumns column logic.
add_roundkey.sv	AddRoundKey operation.	• state_in • round_key	• state_out	Performs a 128-bit combinational XOR between the current state and the selected round key.
key_expansion.sv	Generates the next AES round key.	• key_in • rcon_byte	• key_out	Uses RotWord, SubWord, Rcon, and XOR chaining to generate the next 128-bit round key.

5.2.2 Optimized Module Hierarchy

Table 5.3: Optimized RTL Module Hierarchy

Module / File	Role in Design	Key inputs	Key outputs	Notes
aes128_optimized.sv	Top-level wrapper for the optimized design.	• clk • reset_n • start • key • plaintext	• ciphertext • busy • done	Provides the same external interface as the baseline design and instantiates the optimized aes_core.
aes_core.sv	Optimized AES controller, FSM, and shared-resource datapath.	• clk • reset_n • start • key • plaintext	• ciphertext • busy • done	Contains the main FSM, state register, round counter, shared S-box scheduling, key expansion sequencing, MixColumns sequencing, and output handling.
sbox.sv	AES S-box substitution.	• in_byte	• out_byte	Fixed Boolean logic S-box reused as one shared resource for both KeyExpansion SubWord and AES SubBytes.
shift_rows.sv	AES ShiftRows byte permutation.	• state_in	• state_out	Pure combinational byte rewiring. This block is not shared sequentially because it has low hardware cost.

mix_columns.sv	One-column MixColumns operation.	• col_in	• col_out	Contains a single mix_columns_one_column unit that is reused across the four AES state columns.

5.3 Control

The AES control logic is responsible for sequencing the encryption process from reset and idle condition to final ciphertext generation. The baseline design uses a simpler round-based controller, while the optimized design uses a larger explicit FSM because its datapath resources are shared across multiple cycles.

5.3.1 Baseline Control Flow

The baseline design uses a simple controller based on busy, done, round_reg, and rcon_reg. Since the baseline datapath completes one AES round per clock cycle, it does not require a large multi-state FSM. Instead, the control flow is mainly determined by whether the core is idle, whether encryption is active, and whether the round counter has reached the final AES round.

Table 5.4: Baseline AES Control Flow

State	Purpose	Transition condition	Main outputs / actions
IDLE	Waits for a new AES encryption operation.	A new operation begins when start = 1 and busy = 0.	Performs the initial AddRoundKey by computing plaintext XOR key, sets round_reg = 1, initialises rcon_reg = 8'h01, asserts busy = 1, and clears done.
ROUND	Executes AES rounds 1 to 9.	Active while busy = 1 and round_reg < 10.	Updates state_reg with the AES round output, updates round_key_reg with the next round key, increments round_reg, and updates rcon_reg for the next key expansion step.
FINAL_ROUND	Executes AES round 10.	Occurs when round_reg = 10, causing final_round = 1.	The round datapath performs SubBytes, ShiftRows, and AddRoundKey. MixColumns is bypassed in this stage.
DONE	Stores the final ciphertext and returns the core to IDLE.	Occurs after the final round output is generated.	Stores the final round output into ciphertext, asserts done = 1 for one clock cycle, clears busy, and resets the round control so that the core can accept the next encryption operation.

The baseline control flow is therefore compact because most AES operations are performed combinationally inside the round datapath. The controller mainly tracks the current round, updates the round key, determines when the final round is reached, and generates the busy and done status signals. The simplified baseline control flow is shown in Figure 5.2.

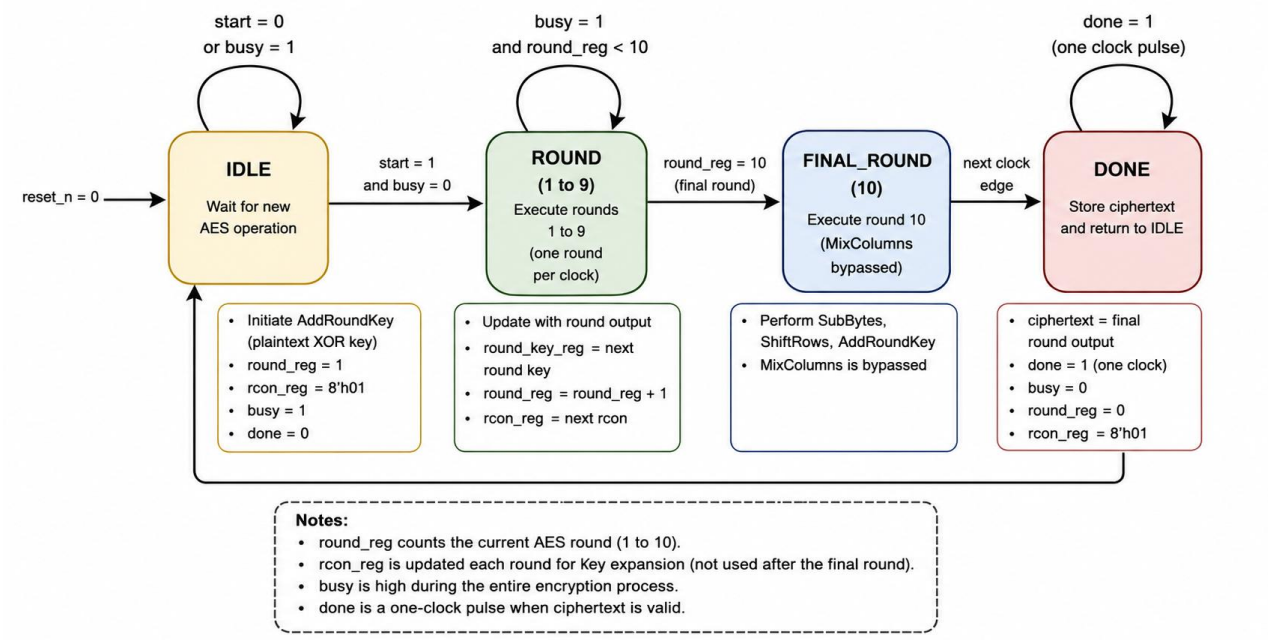

Figure 5.2: Baseline AES Controller – Control Flow Diagram

5.3.2 Optimized Control FSM

The optimized design uses a larger explicit FSM because the AES datapath resources are shared. Instead of substituting all bytes and mixing all columns in parallel, the optimized controller schedules KeyExpansion, SubBytes, ShiftRows, MixColumns, AddRoundKey, and round progression over multiple cycles. This allows one shared S-box and one MixColumns column unit to be reused, reducing hardware area at the cost of additional control states and cycle count.

Table 5.5: Optimized AES Control Flow

State	Purpose	Transition Condition	Main Outputs / Actions
ST_IDLE	Waits for a new encryption request.	Transitions when start is asserted while the core is idle.	Keeps busy = 0 and done = 0. When start is accepted, the plaintext is accepted, the input key is loaded as RoundKey[0], busy is asserted, and the FSM moves to ST_INIT_ADDKEY.
ST_INIT_ADDKEY	Performs the initial AddRoundKey using RoundKey[0].	Completes after one cycle.	Sets state_reg = plaintext XOR key, initialises round_reg = 1, and sets rcon_reg = 8'h01.
ST_KEY_ROTWORD	Prepares the rotated word for key expansion.	Completes after one cycle.	Rotates the previous key word, usually round_key_reg[31:0], into rot_word_reg.
ST_KEY_SUBWORD	Applies SubWord using the shared S-box.	Completes after four bytes have been substituted.	Uses byte_count = 0 to 3 to substitute one byte of rot_word_reg per cycle using the shared S-box.
ST_KEY_MAKE_ROUNDKEY	Generates and stores the next round key.	Completes after one cycle.	Computes w4, w5, w6, and w7 using Rcon and XOR chaining, updates round_key_reg, and prepares the next rcon_reg..
ST_SUBBYTES	Applies AES SubBytes to the state.	Completes after 16 bytes have been substituted.	Uses the same shared S-box and byte_count = 0 to 15 to substitute one state byte per cycle.
ST_SHIFTROWS	Applies ShiftRows.	Completes after one cycle.	Registers the combinational shift_rows_out. If round_reg = 10, the FSM skips MixColumns and moves to ST_ADDROUNDKEY; otherwise, it moves to ST_MIXCOLUMNS.
ST_MIXCOLUMNS	Apply MixColumns one column at a time.	Completes after four columns have been processed.	Uses one mix_columns_one_column block and col_count = 0 to 3 to process the four AES state columns sequentially.
ST_ADDROUNDKEY	XORs the transformed state with the current round key.	If round_reg == 10, transitions to ST_DONE; otherwise, increments the round and returns to key expansion.	Updates the AES state using the current round key. If round_reg = 10, the result is stored as the ciphertext and the FSM moves to ST_DONE. Otherwise, round_reg is incremented and the FSM returns to ST_KEY_ROTWORD to generate the next round key.
ST_DONE	Completes the encryption operation.	Completes after one cycle.	Clears busy, pulses done for one clock cycle, holds the final ciphertext, and returns to ST_IDLE.

The optimized FSM is more complex than the baseline controller because it must control shared hardware resources. The shared S-box is used for both KeyExpansion and SubBytes, while the single MixColumns column unit is reused across four columns. This design reduces duplicated datapath hardware, but it increases the number of cycles required to complete one AES-128 block. The optimized shared-resource FSM is shown in Figure 5.4.

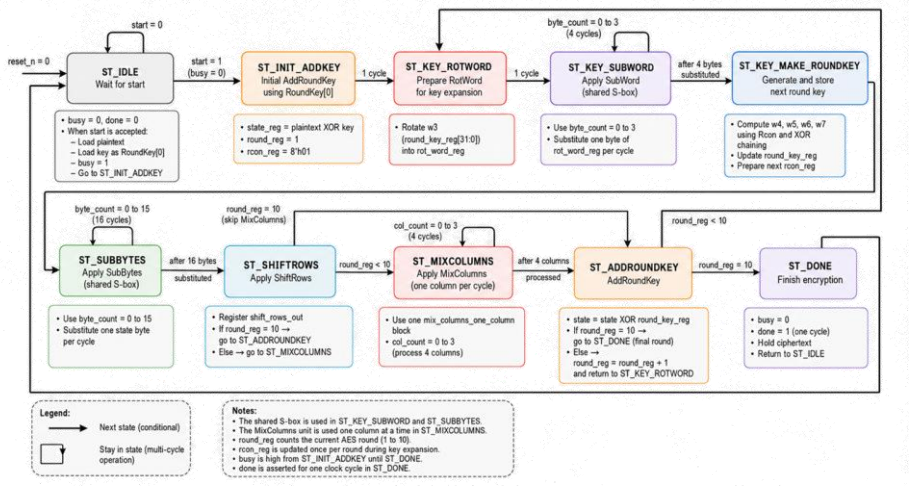

Figure 5.4 Optimized AES Controller – FSM State Diagram

5.4 Datapath Operation and Byte Ordering

The AES datapath operates on a 128-bit internal state. This internal state represents the AES 4 × 4 byte matrix, where the 128-bit input block is arranged column by column. Correct byte ordering is important because SubBytes, ShiftRows, MixColumns, AddRoundKey, and key expansion all depend on consistent byte positions. If the RTL bit slicing does not match the AES column-major state arrangement, the encryption result will not match the expected ciphertext even if the individual AES operations are implemented correctly.

In this design, the most significant byte of the 128-bit state is treated as byte index 0, while the least significant byte is treated as byte index 15. The byte mapping used by the RTL is shown in Table 5.6.

Table 5.6: AES State Byte Ordering Used in the RTL

AES byte index	State matrix position	RTL bit slice	Comment
0	Row 0, Column 0	state[127:120]	First byte of the AES state
1	Row 1, Column 0	state[119:112]	Second byte in column-major order
2	Row 2, Column 0	state[111:104]	Third byte in column-major order
3	Row 3, Column 0	state[103:96]	Fourth byte in column-major order
4	Row 0, Column 1	state[95:88]	First byte of column 1
5	Row 1, Column 1	state[87:80]	Second byte of column 1
6	Row 2, Column 1	state[79:72]	Third byte of column 1
7	Row 3, Column 1	state[71:64]	Fourth byte of column 1
8	Row 0, Column 2	state[63:56]	First byte of column 2
9	Row 1, Column 2	state[55:48]	Second byte of column 2
10	Row 2, Column 2	state[47:40]	Third byte of column 2
11	Row 3, Column 2	state[39:32]	Fourth byte of column 2
12	Row 0, Column 3	state[31:24]	First byte of column 3
13	Row 1, Column 3	state[23:16]	Second byte of column 3
14	Row 2, Column 3	state[15:8]	Third byte of column 3
15	Row 3, Column 3	state[7:0]	Final byte of the AES state

At the beginning of encryption, the initial state is produced by the AddRoundKey operation:

state_reg = plaintext XOR RoundKey[0]

where RoundKey[0] is the original 128-bit input key. After this initial state is created, the datapath updates the state according to the current AES round. For rounds 1 to 9, the state passes through SubBytes, ShiftRows, MixColumns, and AddRoundKey. For round 10, MixColumns is bypassed, and the state passes only through SubBytes, ShiftRows, and AddRoundKey before being stored as the final ciphertext. The general state update sequence is summarised in Table 5.7.

Table 5.7: AES Datapath State Update Sequence

Stage	Datapath Operation	RTL State Behaviour
Initial Step	• plaintext XOR RoundKey[0]	The result is loaded into state_reg.
Rounds 1 to 9	• SubBytes • ShiftRows • MixColumns • AddRoundKey	The transformed round output is written back into state_reg.
Round 10	• SubBytes • ShiftRows • AddRoundKey	MixColumns is bypassed and the final round output is stored as ciphertext.
Completion	• Output Hold	Ciphertext remains stable until reset or until a new encryption operation updates it.

The baseline and optimized designs use the same byte ordering so that both implementations follow the same AES state representation. The difference is how the datapath result is produced. In the baseline design, the full 128-bit state passes through the round datapath in one round operation, so the state register is updated once per AES round. In the optimized design, SubBytes is performed one byte at a time using the shared S-box, while MixColumns is performed one column at a time using the shared mix_columns_one_column unit. The optimized design therefore uses temporary internal storage to collect intermediate byte and column results before updating the full 128-bit state.

This byte-ordering convention is especially important for debugging and waveform inspection. When checking simulation waveforms, the 128-bit state should be interpreted using the byte mapping in Table 5.6. This ensures that changes caused by ShiftRows and MixColumns can be traced correctly against the AES algorithm and the expected ciphertext.

5.5 S-Box Implementation

The AES S-box is implemented as an 8-bit input to 8-bit output substitution block. It is used in both the SubBytes operation in the round datapath and the SubWord operation in key expansion. Since the S-box is one of the most hardware-expensive AES blocks, its implementation style directly affects logic area, timing behaviour, and cycle count.

In the baseline design, the S-box is implemented using a lookup-table style substitution block. The sbox.sv module receives one input byte and produces the corresponding substituted byte. For SubBytes, the baseline sub_bytes.sv module instantiates 16 S-boxes in parallel so that all 16 bytes of the 128-bit AES state can be substituted in the same round operation. Additional S-box usage is also required during key expansion for the SubWord operation when generating the next round key. This approach supports the one-round-per-cycle baseline datapath, but it increases the amount of combinational logic used by the design.

In the optimized design, the S-box is implemented using fixed Boolean logic [5] and reused as one shared S-box resource. Instead of instantiating many S-box blocks, the optimized aes_core.sv schedules S-box access over multiple cycles. During ST_KEY_SUBWORD, the shared S-box substitutes one byte of rot_word_reg per cycle for four cycles. During ST_SUBBYTES, the same S-box substitutes one byte of the AES state per cycle for 16 cycles. The controller uses byte_count to select the current byte and store the substituted output into the correct byte position.

Table 5.8: S-Box Implementation Comparison

Design	S-box Implementation	Usage Method	RTL Behaviour	Effect
Baseline	Lookup-table style S-box	Parallel S-box usage	16 state bytes are substituted in one SubBytes operation. Additional S-box use supports key expansion SubWord.	Lower cycle count but higher logic usage.
Optimized	Fixed Boolean logic S-box	Shared S-box resource	One S-box is reused across KeyExpansion SubWord and SubBytes using byte_count.	Lower duplicated S-box area but higher cycle count.

The baseline and optimized designs therefore implement the same AES substitution function but organise the S-box hardware differently. The baseline design favours lower cycle count by using more parallel S-box resources, while the optimized design favours area reduction by reusing one fixed-Boolean S-box across multiple cycles. This implementation difference is later reflected in the resource and cycle-count comparison.

5.6 MixColumns Implementation

The MixColumns block is used during AES rounds 1 to 9 to mix the bytes within each column of the AES state. In the RTL implementation, MixColumns operates on 32-bit columns, where each column contains four 8-bit state bytes. The operation is implemented using finite-field arithmetic over GF(2^8), mainly through XOR operations and multiplication-by-two logic, commonly represented as xtime.

In the baseline design, the mix_columns.sv module processes the full 128-bit AES state in parallel. The 128-bit state is divided into four 32-bit columns, and each column is passed through a MixColumns column function. This means that all four columns are mixed in the same round operation. The output of the MixColumns block is then passed to AddRoundKey as part of the complete AES round datapath.

Table 5.9: Baseline MixColumns RTL Behaviour

Item	Description
Input Data	128-bit AES state after SubBytes and ShiftRows
Processing method	Four AES columns are processed in parallel
Column Width	32 bits per column
Main logic used	XOR logic and xtime multiplication-by-two function
Output data	128-bit mixed state
Final-round behaviour	Bypassed when final_round is asserted

In the optimized design, MixColumns is implemented using a shared one-column unit. Instead of processing all four columns in parallel, the controller selects one 32-bit column at a time and sends it into the mix_columns_one_column block. The output column is then stored back into the correct position of the temporary state register. This process is controlled using col_count, which counts from 0 to 3 until all four columns have been processed.

Table 5.10: Optimized MixColumns RTL Behaviour

Item	Description
Input Data	128-bit AES state after ShiftRows
Processing method	One 32-bit column is processed per cycle
Column Selector	col_count selects which state column is processed
Shared hardware	One mix_columns_one_column block
Temporary storage	Mixed column outputs are collected into an internal temporary state register
Completion condition	MixColumns completes after four columns have been processed
Final-round behaviour	Skipped when round_reg = 10

The key difference between the two implementations is the amount of MixColumns hardware used. The baseline design uses parallel column processing to support one-round-per-cycle execution, while the optimized design reuses a single column unit to reduce duplicated MixColumns hardware.

During the final AES round, MixColumns is bypassed in both designs. In the baseline design, the round datapath selects the ShiftRows output instead of the MixColumns output before AddRoundKey. In the optimized design, the FSM moves directly from ST_SHIFTROWS to ST_ADDROUNDKEY when round_reg = 10, skipping ST_MIXCOLUMNS.

5.7 Key Expansion Implementation

The key expansion logic generates the round keys required by the AddRoundKey operation. For AES-128, the original 128-bit key is used as RoundKey[0], and the key schedule generates RoundKey[1] to RoundKey[10] for the ten AES encryption rounds. Each round key is 128 bits wide and is formed from four 32-bit words.

In the baseline design, key expansion is implemented as a separate combinational module, key_expansion.sv. The module receives the current 128-bit round key and the current rcon_byte, then generates the next 128-bit round key. The input key is divided into four 32-bit words. The first word of the next round key is generated using RotWord, SubWord, Rcon, and XOR. The remaining three words are generated using XOR chaining.

The baseline key expansion operation can be summarised as follows:

- w4 = w0 XOR SubWord(RotWord(w3)) XOR Rcon
- w5 = w1 XOR w4
- w6 = w2 XOR w5
- w7 = w3 XOR w6
- key_out = {w4, w5, w6, w7}

In this process, Rcon is XORed into the transformed word rather than added using normal arithmetic. This matches the AES key schedule operation and ensures that each round key is different from the previous round key. Since the baseline key expansion is combinational, the next round key can be generated alongside the AES round datapath. Therefore, the baseline design can update both the AES state and the round key once per round cycle.

In the optimized design, key expansion is integrated into the main aes_core.sv FSM instead of being implemented as a separate parallel module. The current round key is stored in round_key_reg, and the key expansion sequence is performed across multiple FSM states. In ST_KEY_ROTWORD, the final word of the current round key is rotated. In ST_KEY_SUBWORD, the shared fixed-Boolean S-box is reused to substitute the four bytes of the rotated word over four cycles. In ST_KEY_MAKE_ROUNDKEY, the controller applies Rcon and XOR chaining to generate the next round key and update round_key_reg.

Table 5.11: Key Expansion Implementation Comparison

Design	Key Expansion Structure	S-Box Usage	Rcon Handling	Effect
Baseline	Separate combinational key_expansion.sv module	Uses S-box logic for SubWord during round-key generation	rcon_byte is updated each round and passed into the key expansion module	Supports one-round-per-cycle operation but uses more parallel combinational logic.
Optimized	Key expansion scheduled inside aes_core.sv FSM	Reuses one shared fixed-Boolean S-box during ST_KEY_SUBWORD	rcon_reg is updated as part of the FSM-controlled key schedule	Reduces duplicated S-box hardware but increases cycle count due to sequential key generation.

The main difference between the two implementations is how much hardware is used to generate the next round key. The baseline design favours faster round-key generation by computing the next key in parallel with the round operation. The optimized design favours area reduction by reusing the shared S-box and spreading key expansion across several FSM states. In both implementations, the generated round keys follow the AES-128 key schedule and are supplied to the AddRoundKey stage according to the current encryption round.

6.0 Functional Verification

Functional verification was carried out using a layered, self-checking simulation strategy. The verification flow was divided into two levels: module-level verification and full-core integration verification. At module level, the main AES transformation blocks were verified using separate testbenches, including SubBytes, ShiftRows, MixColumns, AddRoundKey, key expansion, and AES round-stage testing. After these individual blocks were verified, the complete AES-128 encryption core was tested using a top-level self-checking testbench.

The full encryption testbench verifies one complete AES-128 encryption operation per test vector. Each test vector contains a 128-bit key, a 128-bit plaintext, and the expected 128-bit ciphertext. The vectors are stored in test_vectors.txt using the following format:

Key | PlainText | Expected Ciphertext

Both the baseline and optimized AES-128 designs were verified using the same vector file. This ensures that the comparison between both architectures is fair since both designs are required to produce identical ciphertext outputs for the same reference inputs.

6.1 Testbench Structure

Table 6.1: Testbench features and purpose

Testbench feature	Purpose
Clock and reset generation	A 10 ns clock period is generated continuously. The active-low reset signal reset_n is held low for five clock cycles to initialize the DUT before testing begins.
Start pulse sequencing	The testbench applies the input key and plaintext, waits for one clock edge, and then asserts start for one clock cycle to begin encryption.
Vector loading	The testbench reads the key, plaintext, and expected ciphertext from test_vectors.txt.
Wait-for-done logic	The testbench waits until done == 1 before comparing the DUT ciphertext against the expected ciphertext. This avoids checking the output before encryption has completed.
Timeout protection	MAX_WAIT_CYCLES is set to 10,000 cycles. If done is not asserted within this limit, the testbench reports a timeout failure.
Self-checking comparison	A full 128-bit comparison is performed between ciphertext and expected_ciphertext. The testbench prints the key, plaintext, expected ciphertext, observed ciphertext, cycle count, and PASS/FAIL status.
Idle protection between vectors	Before starting the next vector, the testbench waits until busy is deasserted. This prevents overlapping encryption operations, which is important for FSM-based sequential designs.
Baseline/optimized DUT selection	The same verification method and test vectors are used for both the baseline and optimized designs. The expected cycle count differs because the two architectures have different datapath structures and latencies.

6.2 AES-128 Test Vector Set

The AES-128 encryption vectors used in the full-core verification were obtained from official NIST references. The first group of vectors is taken from the NIST SP 800-38A ECB-AES128 [2] encryption examples. These examples are suitable because ECB mode directly applies the AES block cipher to each 128-bit plaintext block. The second group of vectors is taken from the NIST AES Algorithm Validation Suite Known Answer Test vectors [3], including variable-text and variable-key AES-128 ECB test cases. These vectors check deterministic plaintext and key bit patterns and are widely used for validating AES implementations.

Although Table 6.2 lists ten representative vectors, the full testbench executed a total of 288 AES-128 encryption vectors. This larger set improves confidence that the datapath, key expansion, final round, byte ordering, and control FSM operate correctly across a wide range of inputs.

Table 6.2: Representative AES-128 Test Vectors

ID	Source citation	Plaintext	Key	Expected ciphertext	Observed ciphertext	Pass/Fail
V01	[2]	6bc1bee22e409f96e93d7e117393172a	2b7e151628aed2a6abf7158809cf4f3c	3ad77bb40d7a3660a89ecaf32466ef97	3ad77bb40d7a3660a89ecaf32466ef97	Pass
V02	[2]	ae2d8a571e03ac9c9eb76fac45af8e51	2b7e151628aed2a6abf7158809cf4f3c	f5d3d58503b9699de785895a96fdbaaf	f5d3d58503b9699de785895a96fdbaaf	Pass
V03	[2]	30c81c46a35ce411e5fbc1191a0a52ef	2b7e151628aed2a6abf7158809cf4f3c	43b1cd7f598ece23881b00e3ed030688	43b1cd7f598ece23881b00e3ed030688	Pass
V04	[2]	f69f2445df4f9b17ad2b417be66c3710	2b7e151628aed2a6abf7158809cf4f3c	7b0c785e27e8ad3f8223207104725dd4	7b0c785e27e8ad3f8223207104725dd4	Pass
V05	[3]	80000000000000000000000000000000	00000000000000000000000000000000	3ad78e726c1ec02b7ebfe92b23d9ec34	3ad78e726c1ec02b7ebfe92b23d9ec34	Pass
V06	[3]	c0000000000000000000000000000000	00000000000000000000000000000000	aae5939c8efdf2f04e60b9fe7117b2c2	aae5939c8efdf2f04e60b9fe7117b2c2	Pass
V07	[3]	e0000000000000000000000000000000	00000000000000000000000000000000	f031d4d74f5dcbf39daaf8ca3af6e527	f031d4d74f5dcbf39daaf8ca3af6e527	Pass
V08	[3]	f0000000000000000000000000000000	00000000000000000000000000000000	96d9fd5cc4f07441727df0f33e401a36	96d9fd5cc4f07441727df0f33e401a36	Pass
V09	[3]	f8000000000000000000000000000000	00000000000000000000000000000000	30ccdb044646d7e1f3ccea3dca08b8c0	30ccdb044646d7e1f3ccea3dca08b8c0	Pass
V10	[3]	fc000000000000000000000000000000	00000000000000000000000000000000	16ae4ce5042a67ee8e177b7c587ecc82	16ae4ce5042a67ee8e177b7c587ecc82	Pass

6.3 Simulation Log Evidence

ModelSim simulation logs were generated for both the baseline and optimized AES-128 designs. The logs show that all 288 AES-128 encryption vectors passed for both designs. Each test case prints the input key, plaintext, expected ciphertext, observed ciphertext, cycle count, and PASS/FAIL status. This provides direct evidence that the testbench is self-checking rather than relying only on visual waveform inspection.

The baseline design completed each encryption in a lower number of clock cycles, while the optimized design required a longer cycle count. This difference is expected because the optimized architecture reuses internal hardware resources across multiple cycles to reduce area. Since the testbench waits for the done signal instead of assuming a fixed delay, the same testbench can correctly verify both architectures despite their different latencies.

Further module-level simulation logs were generated for the individual AES blocks, including SubBytes, ShiftRows, MixColumns, AddRoundKey, key expansion, and AES round-stage verification. These logs are included in Appendix B.

```
# ------------------------------------------------
# TEST 286 PASS: AES-128 encryption vector
# KEY        = 2b7e151628aed2a6abf7158809cf4f3c
# PLAINTEXT  = ae2d8a571e03ac9c9eb76fac45af8e51
# EXPECTED   = f5d3d58503b9699de785895a96fdbaaf
# GOT        = f5d3d58503b9699de785895a96fdbaaf
# CYCLES     = 11
# ------------------------------------------------
# TEST 287 PASS: AES-128 encryption vector
# KEY        = 2b7e151628aed2a6abf7158809cf4f3c
# PLAINTEXT  = 30c81c46a35ce411e5fbc1191a0a52ef
# EXPECTED   = 43b1cd7f598ece23881b00e3ed030688
# GOT        = 43b1cd7f598ece23881b00e3ed030688
# CYCLES     = 11
# ------------------------------------------------
# TEST 288 PASS: AES-128 encryption vector
# KEY        = 2b7e151628aed2a6abf7158809cf4f3c
# PLAINTEXT  = f69f2445df4f9b17ad2b417be66c3710
# EXPECTED   = 7b0c785e27e8ad3f8223207104725dd4
# GOT        = 7b0c785e27e8ad3f8223207104725dd4
# CYCLES     = 11
# ================================================
# AES-128 Encryption Testbench Summary
# ================================================
# Total tests : 288
# Passed      : 288
# Failed      : 0
# ================================================
# RESULT: ALL AES-128 TEST VECTORS PASSED
```

Figure 6.1: Baseline AES-128 Verification Log

```
#
# TEST 286 PASS: AES-128 encryption vector
# KEY        = 2b7e151628aed2a6abf7158809cf4f3c
# PLAINTEXT  = ae2d8a571e03ac9c9eb76fac45af8e51
# EXPECTED   = f5d3d58503b9699de785895a96fdbaaf
# GOT        = f5d3d58503b9699de785895a96fdbaaf
# CYCLES     = 279
# ------------------------------------------------
# TEST 287 PASS: AES-128 encryption vector
# KEY        = 2b7e151628aed2a6abf7158809cf4f3c
# PLAINTEXT  = 30c81c46a35ce411e5fbc1191a0a52ef
# EXPECTED   = 43b1cd7f598ece23881b00e3ed030688
# GOT        = 43b1cd7f598ece23881b00e3ed030688
# CYCLES     = 279
# ------------------------------------------------
# TEST 288 PASS: AES-128 encryption vector
# KEY        = 2b7e151628aed2a6abf7158809cf4f3c
# PLAINTEXT  = f69f2445df4f9b17ad2b417be66c3710
# EXPECTED   = 7b0c785e27e8ad3f8223207104725dd4
# GOT        = 7b0c785e27e8ad3f8223207104725dd4
# CYCLES     = 279
# ================================================
# AES-128 Encryption Testbench Summary
# ================================================
# Total tests : 288
# Passed      : 288
# Failed      : 0
# ================================================
# RESULT: ALL AES-128 TEST VECTORS PASSED
```

Figure 6.2: Optimized AES-128 Verification Log

6.4 Waveform Evidence and Debugging Process

Waveform evidence was also generated to verify the timing behaviour of the baseline and optimized AES-128 cores. The waveform screenshots show the reset sequence, input loading, start pulse generation, busy/done handshaking, ciphertext generation, and testbench pass-count update.

At the beginning of simulation, reset_n is held low for five rising clock edges. This initializes the DUT and testbench signals to a known state before encryption begins. After reset is released, the testbench waits for additional clock cycles and confirms that the DUT is idle before applying the key, plaintext, and expected ciphertext for the first test case.

Once the input values are stable, the start signal is asserted for one clock cycle. The AES core then asserts busy, indicating that encryption is in progress. During this period, the testbench does not compare the ciphertext because the done signal has not yet been asserted. The comparison is only performed after done becomes high, ensuring that the ciphertext is valid.

For the baseline design, the waveform shows that the DUT produces the correct ciphertext and asserts done after a shorter number of clock cycles. At completion, busy is deasserted and the ciphertext output is valid. The testbench then compares the observed ciphertext against the expected reference value and increments the pass counter if the values match.

For the optimized design, the same verification sequence is used, but the encryption latency is longer. In the optimized waveform, done is asserted after approximately 279 counted clock cycles. This behaviour is expected because the optimized architecture reuses internal AES resources across multiple cycles instead of using a wider parallel datapath. The correct final ciphertext and PASS result confirm that the optimization preserved functional correctness.

The debugging process was carried out in two stages. First, individual AES modules were verified using separate unit testbenches. The S-box testbench exhaustively checked all 256 possible S-box inputs against the AES reference table. The ShiftRows testbench verified row-shifting behaviour and byte ordering. The MixColumns testbench verified the full 128-bit state transformation and the one-column arithmetic used in the optimized architecture. AddRoundKey verified the XOR operation between the state and round key. The key expansion testbench verified generated round keys against the AES-128 reference key schedule. The AES round-stage testbench verified intermediate stage outputs using the FIPS 197 worked AES-128 example.

After the module-level tests passed, the full AES-128 encryption testbench was used as the integration test. During debugging, the ModelSim waveform and simulation transcript were checked together. The waveform confirmed correct external control sequencing, while the transcript confirmed exact ciphertext matching against reference vectors. If a failure occurred, the printed key, plaintext, expected ciphertext, observed ciphertext, and cycle count would help identify whether the issue came from testbench setup, handshake timing, byte ordering, key expansion, MixColumns, final-round logic, or the top-level FSM. The timeout mechanism also ensured that a design that failed to assert done could not falsely pass.

Overall, the waveform evidence confirms correct AES handshaking and timing behaviour, while the module-level and full-core self-checking testbenches confirm functional correctness at both block level and integration level.

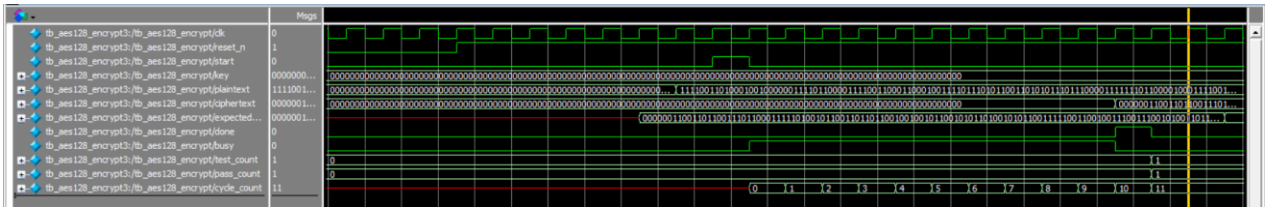

Figure 6.3: Baseline AES-128 Single Test Vector Verification Waveform

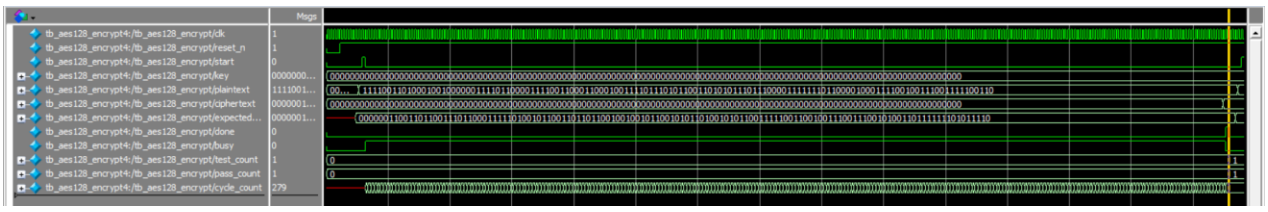

Figure 6.4: Optimized AES-128 Single Test Vector Verification Waveform

7.0 Quartus Compilation and Synthesis Evidence

This section presents the Quartus compilation and synthesis evidence for both the baseline and optimized AES-128 designs. The purpose of this section is to demonstrate that both RTL implementations are synthesizable on the selected FPGA target and to compare their post-synthesis resource utilisation and timing performance. The Quartus results are important because they show the hardware cost of each architecture after compilation, rather than relying only on RTL-level assumptions.

Both designs were compiled using the same FPGA family, target device, and 50 MHz clock constraint. This allows the baseline and optimized designs to be compared under a consistent synthesis setup. The main evidence collected includes the Quartus flow summary, fitter resource utilisation report, TimeQuest timing analysis report, and warning messages. These results confirm that both AES-128 implementations compiled successfully, met the 50 MHz timing constraint, and produced meaningful resource and timing data for later engineering trade-off analysis.

7.1 Quartus Project Configuration

The baseline and optimized AES-128 designs were compiled in Quartus Prime Lite using the same FPGA device family and target device. This ensures that the synthesis results can be compared fairly because both designs were evaluated under the same tool version, FPGA target, and timing model. The Quartus project configuration is summarised in Table 7.1.

Table 7.1: Quartus Project Configuration

Item	Baseline Design	Optimized Design
Quartus Version	Quartus Prime Lite Edition 18.1.0 Build 625	Quartus Prime Lite Edition 18.1.0 Build 625
Target Device Family	MAX 10	MAX 10
Target Device	10M08DAF484C8G	10M08DAF484C8G
Top-Level Entity	aes128_baseline	aes128_optimized
Revision Name	aes128_baseline	aes128_optimized
Clock Frequency Target	50 MHz	50 MHz
Clock Period Constraint	20.000 ns	20.000 ns
SDC Constraint File	aes128_baseline.sdc	aes128_optimized.sdc
RTL Files Included	aes128_baseline.sv aes_core.sv aes_round.sv sub_bytes.sv sbox.sv shift_rows.sv mix_columns.sv add_roundkey.sv key_expansion.sv	aes128_optimized.sv aes_core.sv sbox.sv shift_rows.sv
Compilation Status	Successful	Successful

Both projects completed the Quartus compilation flow successfully. The baseline design uses aes128_baseline as the top-level entity, while the optimized design uses aes128_optimized as the top-level entity. Both designs target the same MAX 10 FPGA device, 10M08DAF484C8G, and use the same 50 MHz clock target. This makes the later resource utilisation and timing comparison meaningful because the differences in results are caused by the AES hardware architecture rather than different project settings.

The Quartus flow summary also confirms that both designs were compiled using the final timing model. Therefore, the reported resource usage and timing results are suitable for comparing the baseline and optimized implementations at synthesis level.

Flow Status	Successful - Wed May 27 14:37:44 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	aes128_baseline
Top-level Entity Name	aes128_baseline
Family	MAX 10
Device	10M08DAF484C8G
Timing Models	Final
Total logic elements	4,943 / 8,064 (61 %)
Total registers	398
Total pins	0 / 250 (0 %)
Total virtual pins	389
Total memory bits	0 / 387,072 (0 %)
Embedded Multiplier 9-bit elements	0 / 48 (0 %)
Total PLLs	0 / 2 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 1 (0 %)

Figure 7.1: Baseline AES-128 Quartus Compilation Flow Summary

Flow Status	Successful - Wed May 27 14:43:04 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	aes_top_optimized
Top-level Entity Name	aes128_optimized
Family	MAX 10
Device	10M08DAF484C8G
Timing Models	Final
Total logic elements	1,463 / 8,064 (18 %)
Total registers	606
Total pins	0 / 250 (0 %)
Total virtual pins	389
Total memory bits	0 / 387,072 (0 %)
Embedded Multiplier 9-bit elements	0 / 48 (0 %)
Total PLLs	0 / 2 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 1 (0 %)

Figure 7.2: Optimized AES-128 Quartus Compilation Flow Summary

7.2 Resource Utilisation Summary

The Analysis & Synthesis stage was completed successfully for both the baseline and optimized AES-128 designs. This stage is important because it confirms that the SystemVerilog RTL code can be converted into synthesizable FPGA logic before fitting, placement, routing, and timing analysis. The baseline design was synthesised using aes128_baseline as the top-level entity, while the optimized design was synthesised using aes128_optimized.

Table 7.2: Quartus Analysis and Synthesis Summary

Item	Baseline Design	Optimized Design	Comment
Analysis & Synthesis status	Successful	Successful	Both RTL designs were accepted by Quartus and successfully converted into FPGA logic.
Top-level entity	aes128_baseline	aes128_optimized	Confirms that the correct top-level wrapper was selected for each design.
Total logic elements	4,941	1,459	The optimized design uses much fewer logic elements because it reuses S-box and MixColumns hardware.
Total registers	398	606	The optimized design uses more registers due to its FSM, byte counter, column counter, and temporary storage.
Total pins	0	0	No physical FPGA pins were assigned because virtual pins were used for synthesis comparison.
Total virtual bits	389	389	Both designs use the same AES top-level interface width.
Total memory bits	0	0	No embedded memory blocks were inferred.
Embedded multiplier 9-bit elements	0	0	No DSP or multiplier resources were required.
Total PLLs	0	0	No PLL was used in either design.
UFM blocks	0	0	User flash memory was not used.
ADC blocks	0	0	ADC resources were not used.

The Analysis & Synthesis results show that both AES-128 designs are synthesizable and suitable for further compilation stages. The optimized design reduces total logic elements from 4,941 to 1,459, which is consistent with the intended area-focused architecture. This reduction is mainly due to shared hardware resources, especially the shared S-box and one-column MixColumns implementation.

However, the optimized design increases register usage from 398 to 606. This is expected because the optimized design performs AES operations across more FSM states and requires additional counters and temporary registers to store intermediate byte and column results. Therefore, the synthesis result already shows the main design trade-off: the optimized design reduces combinational logic area but requires more sequential control storage..

Analysis & Synthesis Status	Successful - Wed May 27 14:36:59 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	aes128_baseline
Top-level Entity Name	aes128_baseline
Family	MAX 10
Total logic elements	4,941
Total registers	398
Total pins	0
Total virtual pins	389
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0
UFM blocks	0
ADC blocks	0

Figure 7.3: Baseline AES-128 Quartus Synthesis Summary

Analysis & Synthesis Status	Successful - Wed May 27 14:42:33 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	aes_top_optimized
Top-level Entity Name	aes128_optimized
Family	MAX 10
Total logic elements	1,459
Total registers	606
Total pins	0
Total virtual pins	389
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0
UFM blocks	0
ADC blocks	0

Figure 7.4: Optimized AES-128 Quartus Synthesis Summary

7.3 Resource Utilisation Summary

The Quartus resource utilisation results are summarised in Table 7.3. These results compare the amount of FPGA hardware required by the baseline and optimized AES-128 designs after synthesis and fitting. The optimized design uses significantly fewer logic elements than the baseline design, showing that the area reduction objective was achieved. However, the optimized design uses more registers because the shared-resource architecture requires additional FSM states, byte counters, column counters, and temporary storage.

Table 7.3: Quartus Resource Utilisation Summary

Metric	Baseline Design	Optimized Design	Difference	Comment
Total Logic Elements	4,941	1,459	3,482 fewer about 70.5% reduction	Optimized design reduces logic area significantly by sharing S-box and MixColumns hardware instead of duplicating parallel datapath logic.
Total combinational functions	4,941	1,458	3,483 fewer about 70.5% reduction	Large reduction shows that most area saving comes from reducing combinational AES logic, especially S-box and MixColumns logic.
Total Registers	398	606	208 more about 52.3% increase	Optimized design uses more registers because it needs FSM states, byte/column counters, and temporary storage for multi-cycle operation.
Dedicated logic registers	398	606	208 more about 52.3% increase	Register increase is expected because shared-resource execution requires intermediate data to be stored across cycles.
Virtual Pins	389	389	No difference	Same for both designs because both use the same AES top-level interface.
I/O Pins	0	0	No difference	Zero physical I/O pins because virtual pins were used for synthesis comparison.
Embedded multiplier 9-bit elements	0	0	No difference	No DSP or multiplier blocks are used; AES operations are implemented using logic and XOR operations.
Maximum fan-out node	clk	clk	Same	clk has the highest fan-out because it drives the sequential registers.
Maximum fan-out	398	606	208 more about 52.3% increase	Higher in the optimized design because it has more registers.
Total fan-out	21,210	7,570	13,640 fewer about 64.3% reduction	Lower in the optimized design because it has less parallel combinational logic.
Average fan-out	3.70	3.09	0.61 lower about 16.5% reduction	Lower average fan-out suggests reduced overall signal connectivity in the optimized design.

The logic element reduction of 70.5% has confirmed that the shared-resource optimization successfully reduces logic area. The reduction mainly comes from reusing one S-box resource and one MixColumns column unit across multiple cycles instead of implementing more parallel AES datapath hardware. However, the register count increases from 398 to 606 because the optimized design requires additional FSM control, counters, and temporary storage for sequential byte-level and column-level processing. Therefore, the Quartus resource results show the intended area-control trade-off: the optimized design uses much less combinational logic but requires more sequential control storage.

	Resource	Usage
1	Estimated Total logic elements	4,941
2		
3	Total combinational functions	4941
4	Logic element usage by number of LUT inputs	
1	-- 4 input functions	4722
2	-- 3 input functions	163
3	-- <=2 input functions	56
5		
6	Logic elements by mode	
1	-- normal mode	4941
2	-- arithmetic mode	0
7		
8	Total registers	398
1	-- Dedicated logic registers	398
2	-- I/O registers	0
9		
10	Virtual pins	389
11	I/O pins	0
12		
13	Embedded Multiplier 9-bit elements	0
14		
15	Maximum fan-out node	clk
16	Maximum fan-out	398
17	Total fan-out	21210
18	Average fan-out	3.70

Figure 7.5: Baseline AES-128 Quartus Resource Utilisation Summary

	Resource	Usage
1	Estimated Total logic elements	1,459
2		
3	Total combinational functions	1458
4	Logic element usage by number of LUT inputs	
1	-- 4 input functions	919
2	-- 3 input functions	320
3	-- <=2 input functions	219
5		
6	Logic elements by mode	
1	-- normal mode	1458
2	-- arithmetic mode	0
7		
8	Total registers	606
1	-- Dedicated logic registers	606
2	-- I/O registers	0
9		
10	Virtual pins	389
11	I/O pins	0
12		
13	Embedded Multiplier 9-bit elements	0
14		
15	Maximum fan-out node	clk
16	Maximum fan-out	606
17	Total fan-out	7570
18	Average fan-out	3.09

Figure 7.6: Optimized AES-128 Quartus Resource Utilisation Summary

7.4 Timing Summary

The Quartus TimeQuest Timing Analyzer was used to evaluate whether the baseline and optimized AES-128 designs meet the required 50 MHz clock target. A 20.000 ns clock period constraint was applied to both designs. The timing results are summarised in Table 7.4.

Table 7.4: Quartus Timing Summary

Timing metric	Baseline	Optimized	Interpretation
Target clock period	20.000 ns	20.000 ns	Both designs were constrained to a 50 MHz clock target.
Target clock frequency	50.0 MHz	50.0 MHz	Same timing target was used for fair comparison.
Achieved Fmax	72.23 MHz	57.84 MHz	Both designs exceed the required 50 MHz clock frequency.
Worst setup slack	6.156 ns	2.710 ns	Positive setup slack means the setup timing requirement is met.
Worst hold slack	0.150 ns	0.150 ns	Positive hold slack means the hold timing requirement is met.
End Point TNS	0.000 ns	0.000 ns	No total negative slack was reported.
Expected timing contributor based on RTL structure	AES round datapath, including S-box, MixColumns, and key expansion logic	Shared S-box path, FSM-controlled datapath selection, state multiplexing, and resource-sharing paths	The detailed TimeQuest path report should be used if exact critical path identification is required.

Both designs meet the 50 MHz timing target because the achieved Fmax values are higher than 50 MHz and the reported setup and hold slacks are positive. The baseline design achieves a higher Fmax of 72.23 MHz, while the optimized design achieves 57.84 MHz. This means that the baseline design has more timing margin under the selected 20 ns clock period.

Although the baseline design uses more logic elements, its datapath structure is more direct and its control logic is simpler. The optimized design still passes timing, but its lower Fmax suggests that the shared-resource architecture introduces extra timing overhead from FSM-controlled selection, byte-level processing, column-level processing, and internal multiplexing. Therefore, the optimized design successfully reduces area, but with lower maximum operating frequency compared with the baseline design.

Since the worst setup slack and worst hold slack are both positive for the baseline and optimized designs, neither implementation violates the 20 ns clock constraint. The timing results therefore confirm that both AES-128 designs are timing-clean for the selected 50 MHz target.

Clock Information

#	Clock Name	Type	Period	Frequency	Rise	Fall
1	clk	Base	20.000	50.0 MHz	0.000	10.000

Fmax Summary

#	Fmax	Restricted Fmax	Clock Name	Note
1	72.23 MHz	72.23 MHz	clk	

Setup Summary (Worst Setup Slack)

#	Clock	Slack	End Point TNS
1	clk	6.156	0.000

Hold Summary (Worst Hold Slack)

#	Clock	Slack	End Point TNS
1	clk	0.150	0.000

Figure 7.7: Baseline AES-128 Quartus Timing Analyzer Summary

Clock Information

#	Clock Name	Type	Period	Frequency	Rise	Fall
1	clk	Base	20.000	50.0 MHz	0.000	10.000

Fmax Summary

#	Fmax	Restricted Fmax	Clock Name	Note
1	57.84 MHz	57.84 MHz	clk	

Setup Summary (Worst Setup Slack)

#	Clock	Slack	End Point TNS
1	clk	2.710	0.000

Hold Summary (Worst Hold Slack)

#	Clock	Slack	End Point TNS
1	clk	0.150	0.000

Figure 7.8: Optimized AES-128 Quartus Timing Analyzer Summary

7.5 Quartus Evidence Summary

The Quartus compilation evidence confirms that both the baseline and optimized AES-128 designs were successfully compiled for the same MAX 10 target device, 10M08DAF484C8G, using Quartus Prime Lite Edition 18.1.0. Both designs used the same 50 MHz timing target with a 20.000 ns clock period constraint, allowing the resource and timing results to be compared fairly.

The Analysis & Synthesis results show that both RTL designs were successfully converted into synthesizable FPGA logic. The baseline design used 4,941 logic elements and 398 registers, while the optimized design used 1,459 logic elements and 606 registers. This shows that the optimized design significantly reduced logic usage but required more sequential storage due to its FSM-controlled shared-resource architecture.

The resource utilisation results further support the intended optimization objective. The optimized design reduced estimated total logic elements from 4,941 to 1,459, corresponding to approximately 70.5% reduction. This reduction mainly comes from reusing one S-box resource and one MixColumns column unit instead of implementing more parallel AES datapath hardware. However, the register count increased from 398 to 606 because the optimized design requires additional FSM states, byte counters, column counters, and temporary registers.

The TimeQuest timing results show that both designs met the 50 MHz timing requirement. The baseline design achieved an Fmax of 72.23 MHz, while the optimized design achieved an Fmax of 57.84 MHz. Both values are above the 50 MHz target. The baseline design also had a larger setup slack of 6.156 ns, compared with 2.710 ns for the optimized design. Therefore, both implementations are timing-clean, but the baseline design has greater timing margin.

The Quartus warnings were reviewed and documented. The virtual clock and virtual pin warnings were caused by compiling the AES designs using virtual pins instead of physical FPGA pin assignments. These warnings do not affect RTL functional correctness, but they should be addressed for a final board-level implementation. The baseline S-box warning was also reviewed against the simulation evidence, and the full AES testbench results confirmed that the design still produced the correct ciphertext.

Overall, the Quartus evidence demonstrates successful compilation, valid synthesis, meaningful resource comparison, and successful timing closure for both AES-128 implementations. The results support the main engineering trade-off of the project: the baseline design uses more logic resources but achieves higher Fmax and lower cycle count, while the optimized design greatly reduces logic area at the cost of more registers, lower Fmax, and higher cycle count.

8.0 Tiny Tapeout Backend Workflow Evidence

This section presents the Tiny Tapeout backend workflow evidence for both the baseline and optimized AES-128 designs. The purpose of this section is to show that the RTL designs were not only functionally correct in simulation but were also processed through an ASIC-style backend flow to generate physical implementation evidence.

Both designs were adapted from the original Quartus/FPGA-style RTL into Tiny Tapeout-compatible ASIC submission projects using the official Tiny Tapeout SKY Verilog template [4]. The optimized AES-128 design was selected as the intended final Tiny Tapeout submission because it provides a more area-efficient architecture. However, the baseline design was also processed through the same backend workflow to provide a fair physical implementation comparison.

Minor wrapper-level changes were required because the original AES cores use 128-bit key, plaintext, and ciphertext ports, while the Tiny Tapeout user module has limited external I/O pins. Therefore, both designs were wrapped with an 8-bit byte-serial interface. This allows the 128-bit key and plaintext to be loaded over multiple cycles and the 128-bit ciphertext to be read out one byte at a time.

8.1 Submission Preparation

Table 8.1 summarises the Tiny Tapeout submission preparation for both the baseline and optimized AES-128 designs. Each design was placed in a separate GitHub repository and configured independently using the Tiny Tapeout SKY Verilog template.

Table 8.1: Submission Items for Tiny Tapeout SKY Verilog

Item	Description / Evidence (Baseline)	Description / Evidence (Optimized)
GitHub repository	https://github.com/xcore11/ttsky-aes128-baseline	https://github.com/xcore11/ttsky-aes128-optimized
Tiny Tapeout template	Tiny Tapeout SKY Verilog template	Tiny Tapeout SKY Verilog template
Tiny Tapeout top module	tt_um_aes128_baseline	tt_um_aes128_optimized
Final successful tile allocation	8x2	4x2
Input data port	ui_in[7:0] used as 8-bit input byte	ui_in[7:0] used as 8-bit input byte
Output data port	uo_out[7:0] used as 8-bit ciphertext output byte	uo_out[7:0] used as 8-bit ciphertext output byte
Control inputs	uio_in[0] = load_key uio_in[1] = load_plaintext uio_in[2] = start uio_in[3] = read_ciphertext_next uio_in[4] = clear	uio_in[0] = load_key uio_in[1] = load_plaintext uio_in[2] = start uio_in[3] = read_ciphertext_next uio_in[4] = clear
Status outputs	uio_out[5] = busy uio_out[6] = done uio_out[7] = ready	uio_out[5] = busy uio_out[6] = done uio_out[7] = ready
Main files modified	• info.yaml • src/ • test/Makefile • test/tb.v • test/test.py • docs/info.md	• info.yaml • src/ • test/Makefile • test/tb.v • test/test.py • docs/info.md
Test method	Python/Tiny Tapeout testbench loads AES key and plaintext byte-by-byte, starts encryption, then reads out ciphertext byte-by-byte	Python/Tiny Tapeout testbench loads AES key and plaintext byte-by-byte, starts encryption, then reads out ciphertext byte-by-byte.

The wrapper converts the original AES-128 interface into a Tiny Tapeout-compatible byte-serial interface. The key is loaded as 16 individual bytes using load_key, and the plaintext is loaded as 16 individual bytes using load_plaintext. After both 128-bit inputs are available internally, the start control signal begins encryption. When encryption completes, the done signal is asserted and the ciphertext is read out through uo_out[7:0] using 16 read_ciphertext_next pulses. This approach preserves the functional behaviour of the original AES core while satisfying the limited I/O constraints of the Tiny Tapeout platform.

8.2 Hardening Workflow Results

The Tiny Tapeout backend workflow was executed through GitHub Actions. The workflow generated project documentation, ran functional tests, performed Tiny Tapeout precheck validation, and hardened the RTL into a physical GDS layout. Among these stages, the GDS workflow provides the most important backend evidence because it confirms that the design could be synthesized, placed, routed, checked, and packaged for Tiny Tapeout submission.

For the optimized design, the first hardening attempt using a 1x1 tile allocation failed because the reported utilization was approximately 409.596%, which greatly exceeded the available placement area. This showed that the optimized AES-128 design could not physically fit into a single Tiny Tapeout tile. After increasing the optimized tile allocation to 4x2, the GDS workflow completed successfully. The baseline design required an even larger final tile allocation of 8x2 to complete backend hardening successfully.

Figure 8.2 shows the successful GitHub Actions workflow result, including the GDS and precheck stages. This confirms that the backend flow completed without fatal errors after the tile allocation was adjusted. In addition to the workflow status, Tiny Tapeout also generated several backend artifacts, which provide more detailed evidence of the hardening result. These artifacts are summarised in Table 8.2.

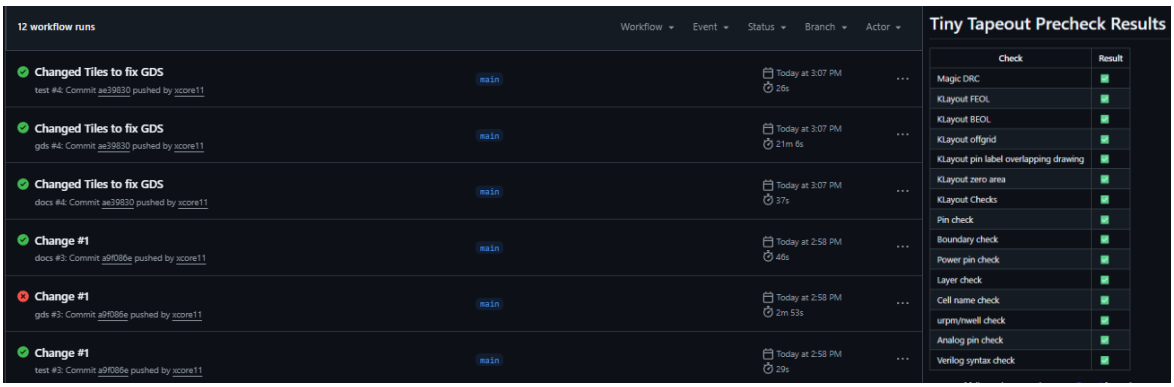

Figure 8.2: Evidence of GDS Success and Precheck Results

Table 8.2: Tiny Tapeout Backend Artifacts

Artifact Name	Uses / Functionality
gds_render	Provides the rendered layout image of the hardened design.
tt_submission	Contains the final Tiny Tapeout submission package.
GDS_log	Contains backend hardening logs and OpenROAD flow evidence.
precheck_reports	Contains Tiny Tapeout precheck results for submission rule compliance.
gatelevel_test_results	Confirms that the hardened gate-level design still passes functional testing.
github-pages	Contains generated documentation files for the project page.

Together, Figure 8.2 and Table 8.2 show that the backend evidence is not limited to a visual screenshot. The successful workflow status confirms that the backend process completed, while the generated artifacts provide the layout render, submission package, hardening logs, precheck reports, and gate-level verification results. This provides stronger evidence that both AES-128 designs were successfully processed through the Tiny Tapeout backend flow.

8.2.1 Baseline GDS Results

Table 8.3: Parameters of Baseline GDS Results

Parameters	Number
Tiles	8x2
Area Utilization (%)	43.644 %
Wire Length (um)	726069
Fill Count	46063
Combo Logic Count	5294
Tap Count	4399
Clock Count	1551
NOR gate Count	1165
OR gate Count	947
NAND gate Count	856
Flip Flops Count	798
Buffer Count	643
Multiplexer Count	553
AND gate Count	395
Diode Count	347
Misc Count	262
Inverter Count	165

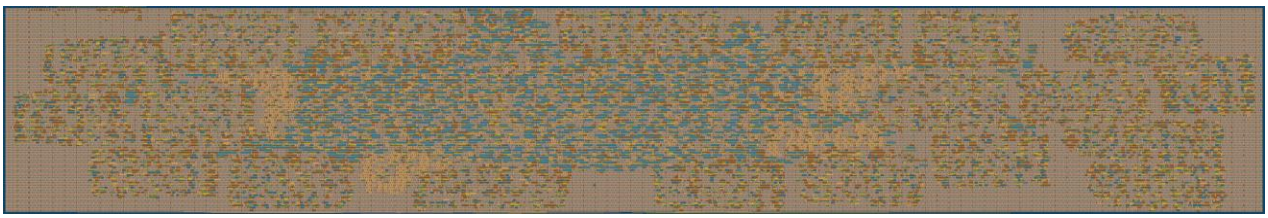

Figure 8.3. Generated Layout Render for Baseline Version

The baseline design completed the Tiny Tapeout backend flow successfully, but it required a larger 8x2 tile allocation. This is expected because the baseline architecture uses a more parallel datapath with more combinational AES hardware. Although this allows lower encryption latency, it increases physical area and routing demand.

8.2.2 Optimized GDS Results

Table 8.4: Parameters of Optimized GDS Results

Parameters	Number
Tiles	4x2
Area Utilization (%)	44.698 %
Wire Length (um)	259031
Fill count	20403
Tap count	2158
Combo logic count	1444
Flip Flops count	1010
Misc Count	733
Multiplexer Count	731
Clock Count	540
Buffer Count	479
OR gate Count	324
NOR gate Count	282
NAND gate Count	140
Diode Count	100
AND gate Count	76
Inverter Count	51

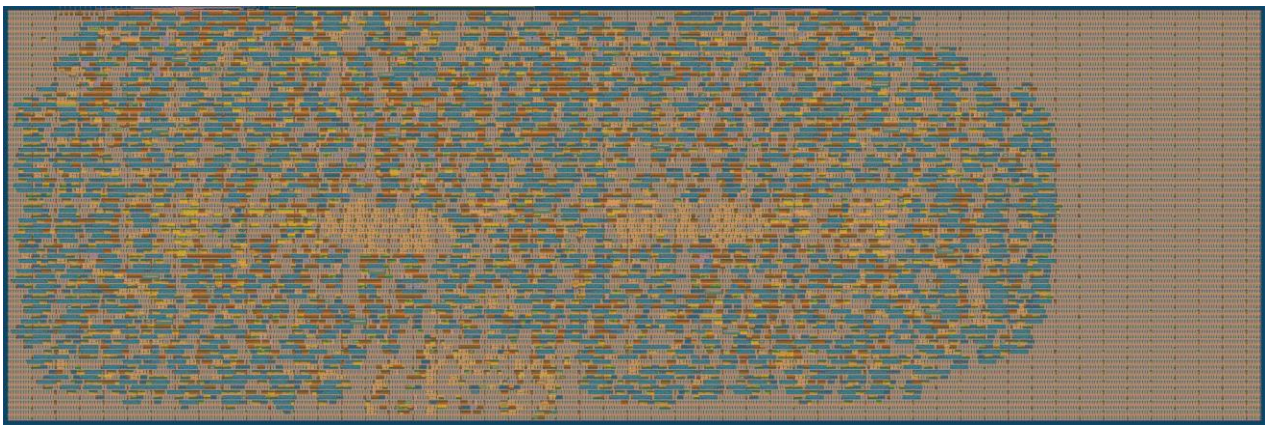

Figure 8.4. Generated Layout Render for Optimized Version

The optimized design also successfully completed backend hardening using a 4x2 tile allocation. This demonstrates that the optimized architecture is physically more compact and more suitable for the area-constrained Tiny Tapeout backend flow. Its reduced combinational logic and shorter wire length indicate that the resource-sharing approach effectively reduces layout complexity.

8.3 Backend Evidence Discussion

The Tiny Tapeout backend results show a clear physical implementation difference between the baseline and optimized AES-128 designs. Both designs successfully completed the backend workflow, but the optimized design required only a 4x2 tile allocation, while the baseline required 8x2. Since 8x2 provides approximately twice the tile footprint of 4x2, the baseline design requires significantly more physical area even though the final utilization percentages appear similar.

The area utilization values alone should not be interpreted without considering tile allocation. The optimized design reports 44.698% utilization inside a 4x2 region, while the baseline reports 43.644% utilization inside a larger 8x2 region. Therefore, the similar utilization percentages do not mean the two designs have similar area. Instead, the baseline achieves a similar percentage only because it is placed in a much larger physical region. This confirms that the optimized architecture has a substantially smaller backend footprint.

The routing results further support this conclusion. The baseline design has a total wire length of 726,069 µm, while the optimized design has only 259,031 µm. This means the baseline requires approximately 2.8 times the routing length of the optimized design. Equivalently, the optimized design reduces total wire length by approximately 64.3% compared with the baseline. This is a significant backend improvement because shorter routing generally reduces routing congestion, parasitic capacitance, and dynamic switching power.

The standard-cell breakdown also confirms the effectiveness of the optimized architecture. The baseline design contains 5,294 combinational logic cells, while the optimized design contains only 1,444. This represents an approximate 72.7% reduction in combinational logic. The baseline also has much higher individual logic-gate counts, including 1,165 NOR gates, 947 OR gates, and 856 NAND gates, compared with 282 NOR gates, 324 OR gates, and 140 NAND gates in the optimized design. This shows that the optimized design successfully reduces the amount of duplicated combinational AES hardware.

However, the optimized design is not smaller in every category. It contains 1,010 flip-flops compared with 798 in the baseline, and 731 multiplexers compared with 553 in the baseline. This is an expected trade-off for an area-optimized sequential architecture. Instead of using a wide parallel datapath, the optimized design reuses internal AES resources over more cycles. This reduces combinational logic and routing demand, but requires additional registers, multiplexers, and FSM control to sequence the reused datapath.

This backend result also matches the functional verification results. The optimized design has a longer encryption latency than the baseline, but it achieves a much smaller physical implementation. Therefore, the optimization is a clear area-latency trade-off: the optimized architecture sacrifices speed by requiring more cycles per encryption, but significantly reduces combinational logic, routing length, and tile requirement.

Overall, the Tiny Tapeout backend evidence confirms that the optimized AES-128 design is more suitable for an area-constrained ASIC implementation. It uses half the tile allocation of the baseline, reduces total wire length by approximately 64.3%, and reduces combinational logic count by approximately 72.7%. The baseline design remains functionally correct and backend-valid, but it requires a larger physical footprint and substantially more routing resources. This supports the project objective of producing an AES-128 hardware implementation that maintains correctness while improving area efficiency.

9.0 Performance Comparison and Optimization Analysis

This section compares the baseline and optimized AES-128 implementations using functional verification results, timing results, FPGA resource reports, and Tiny Tapeout backend physical evidence. The comparison is fair because both designs implement the same AES-128 encryption algorithm, use the same start/busy/done control concept, and were verified using the same self-checking AES-128 testbench and reference test vectors described in Section 6.

The main difference between the two designs is architectural. The baseline design prioritizes lower latency by using a more parallel AES datapath, while the optimized design prioritizes smaller hardware area by reusing internal resources across multiple cycles. Therefore, the comparison is not only based on speed, but also on the area, routing, timing, and backend implementation trade-offs introduced by the optimization.

9.1 Baseline vs Optimized Summary

Table 9.1 summarizes the major architectural, FPGA, timing, and backend differences between the baseline and optimized AES-128 implementations.

Table 9.1. Baseline and Optimized AES-128 Comparison Summary

Comparison item	Baseline implementation	Optimized implementation	Measured effect	Engineering explanation
Architecture	One-round-per-cycle iterative AES datapath.	Shared-resource multi-cycle AES datapath.	11 cycles vs 279 cycles. (*Section 6.3*)	The baseline completes one AES round per cycle using more parallel logic. The optimized design reuses hardware over many cycles, increasing latency but reducing area.
S-box approach	Multiple lookup-table S-box instances.	One shared fixed-Boolean S-box.	Lower duplicated logic in the optimized design.	As discussed in *Section 5.5*, the optimized design reuses one S-box for both SubBytes and key expansion SubWord, reducing repeated S-box hardware.
Key expansion	Separate combinational key expansion module.	FSM-controlled key expansion using the shared S-box.	Lower area, higher cycle count.	The optimized design performs RotWord, SubWord, Rcon, and XOR chaining over multiple FSM states instead of computing the next round key in parallel.
MixColumns	Four columns processed in parallel.	One column processed per cycle.	Reduced combinational logic and routing.	*Section 5.6* shows that the optimized design reuses one MixColumns column unit, while the baseline processes all four columns in parallel.
Backend area	8x2 Tiny Tapeout tile allocation.	4x2 Tiny Tapeout tile allocation.	50% tile-footprint reduction.	*Section 8.2* confirms that the optimized design fits in half the Tiny Tapeout tile area required by the baseline.
Backend routing	726,069 um wire length.	259,031 um wire length.	64.3% wire-length reduction.	*Section 8.2* confirmed the optimized layout has much lower routing demand, indicating reduced physical complexity and better backend suitability.
Standard-cell logic	5,294 combinational cells.	1,444 combinational cells.	72.7% reduction.	*Section 8.2* confirms the optimized design successfully removes large amounts of duplicated AES datapath logic.
Control overhead	798 flip-flops and 553 multiplexers.	1,010 flip-flops and 731 multiplexers.	FF +26.6%, MUX +32.2%.	*Section 8.2* confirms the optimized design uses more registers and multiplexers because resource sharing requires extra FSM control, temporary storage, and byte/column selection logic.
Quartus logic elements	4,941	1,459	3,482 fewer, about 70.5% reduction	*Section 7.2* confirms that resource sharing significantly reduces FPGA logic area.
Quartus registers	398	606	208 more, about 52.3% increase	*Section 7.2* confirms the optimized design needs more FSM states, byte counters, column counters, and temporary registers.
Quartus total fan-out	21,210	7,570	13,640 fewer, about 64.3% reduction	*Section 7.3* confirms the optimized design has less parallel datapath connectivity and lower overall signal distribution.
Fmax	72.23 MHz	57.84 MHz	Optimized is 14.39 MHz lower	*Section 7.4* confirms the optimized design still meets 50 MHz, but its FSM-controlled multiplexing and shared-resource paths reduce maximum frequency.
Setup slack	6.156 ns	2.710 ns	Both positive	*Section 7.4* confirms both designs meet setup timing at the 20 ns clock constraint. The baseline has more timing margin.
Hold slack	0.150 ns	0.150 ns	Both positive	*Section 7.4* confirms both designs meet hold timing.

9.2 Latency and Throughput Calculation

Latency is calculated by multiplying the measured cycle count by the target clock period. Since both designs are evaluated using the same 50 MHz target clock, the clock period is 20 ns. Throughput is then calculated by dividing the AES block size, 128 bits, by the encryption latency.

Table 9.2 Latency and Throughput calculation and evaluation

Metric	Formula / meaning	Baseline	Optimized	Comment
Cycle count	Cycles from accepted start to done.	11 cycles	279 cycles	Taken from the waveform discussion in *Section 6.4*.
Clock period	50 MHz target clock.	20 ns	20 ns	Same clock target from the Tiny Tapeout configuration in *Section 8.1*.
Latency	Cycle count x clock period.	200 ns	5.58 us	The baseline is much faster per block because it uses more parallel AES hardware.
Throughput	128 bits / latency.	640 Mbps	22.9 Mbps	The baseline has approximately 27.9x higher throughput at the same clock.

The calculation shows that the baseline design is significantly faster in terms of per-block latency and throughput. However, this speed is achieved by using more parallel hardware, which increases logic area and routing demand. The optimized design sacrifices throughput in exchange for much lower physical resource usage.

9.3 Area, Timing, and Latency Trade-Offs

The results demonstrate a clear and intentional area-latency trade-off. The baseline design achieves low latency because it uses a more parallel AES datapath. It completes one encryption block in 11 cycles, reaches an Fmax of 72.23 MHz, and provides a throughput of 640 Mbps at the 50 MHz target clock. This makes the baseline design more suitable when speed or throughput is the main design priority. However, this performance comes at a high hardware cost. The baseline uses 4,941 Quartus logic elements, 5,294 Tiny Tapeout combinational cells, and 726,069 µm of backend wire length.

The optimized design makes the opposite architectural choice. It reduces duplicated AES hardware by reusing one S-box and one MixColumns column unit across multiple cycles. As a result, the Quartus logic element count decreases from 4,941 to 1,459, which is approximately a 70.5% reduction. The Tiny Tapeout combinational cell count also decreases from 5,294 to 1,444, which is approximately a 72.7% reduction. Backend wire length is reduced from 726,069 µm to 259,031 µm, and the required Tiny Tapeout tile allocation decreases from 8x2 to 4x2. These improvements directly support the project objective of producing a smaller AES-128 hardware implementation.

The cost of this optimization is increased control complexity and longer latency. The optimized design requires 279 cycles per encryption block, compared with only 11 cycles for the baseline. Its Quartus register count also increases from 398 to 606 because the shared-resource architecture requires FSM state registers, byte counters, column counters, and temporary storage. Similarly, the Tiny Tapeout backend results show an increase in flip-flops and multiplexers. This is expected because resource sharing replaces large parallel combinational datapaths with sequential control, temporary registers, and selection logic.

The timing results also support this interpretation. The baseline reaches a higher Fmax of 72.23 MHz, while the optimized design reaches 57.84 MHz. This indicates that the optimized design's FSM-controlled multiplexing and shared-resource paths introduce additional timing overhead. However, both designs still meet the 50 MHz target clock, with positive setup slack and positive hold slack. Therefore, the optimized design does not fail timing; it simply has less timing margin than the baseline.

The backend results are especially important because this project targets a Tiny Tapeout-style ASIC implementation. In this context, silicon area and routing complexity are major constraints. Although the baseline design is faster, it requires twice the Tiny Tapeout tile allocation and approximately 2.8 times the backend wire length. In contrast, the optimized design fits into a smaller physical footprint and significantly reduces combinational logic and routing demand. This makes the optimized design more suitable for an area-constrained ASIC backend flow.

Overall, the optimized AES-128 implementation is the better choice for this project because it achieves the main optimization goal: reducing hardware area and backend physical complexity while preserving correct AES-128 functionality. The final comparison shows a meaningful engineering trade-off. The baseline design is preferable for high-throughput applications, while the optimized design is preferable for compact silicon implementation. Since this project includes Tiny Tapeout backend hardening, the optimized design is the stronger final submission candidate.

10.0 Engineering Discussion, Limitations, and Future Improvements

This section discusses the engineering lessons learned from the AES-128 hardware accelerator design. The project compared a baseline implementation and an optimized implementation to evaluate how architectural choices affect area, timing, latency, and implementation complexity. The baseline design was developed as the reference implementation, while the optimized design was developed to reduce hardware area through resource sharing. Both designs produced the correct AES-128 ciphertext and successfully compiled in Quartus, but the hardware results show that different RTL organisations lead to different trade-offs.

The baseline design represents a straightforward AES-128 hardware implementation with a more direct datapath and simpler control structure. It does not specifically minimise area or cycle count, but provides a reliable reference point for comparison. In contrast, the optimized design prioritises lower logic area by reusing selected hardware resources across multiple cycles. The Quartus results show that the optimized design reduced total logic elements from 4,941 to 1,459, which is approximately 70.5% reduction. However, this area improvement came with higher register usage, lower Fmax, and a much higher cycle count. Therefore, the project demonstrates that hardware optimization is not simply about improving one metric, but about balancing area, timing, latency, and design complexity.

10.1 Engineering Trade-Offs

The main engineering trade-off in this project is between parallel execution and resource sharing. The baseline AES-128 design provides a straightforward reference implementation with a more direct datapath. As a result, it uses more parallel logic than the optimized design, including parallel S-box substitution and full-state MixColumns processing. This allows the design to complete encryption in fewer cycles, with the baseline design completing one encryption block in 11 cycles. However, this approach uses more logic elements because multiple AES operations are implemented in parallel.

The optimized design reduces duplicated datapath hardware by sharing one fixed-Boolean S-box and one MixColumns column unit across multiple cycles. This significantly reduces combinational logic usage, as shown by the reduction from 4,941 to 1,459 total logic elements. However, the optimized design requires additional FSM states, counters, temporary registers, and sequential control logic. This increases the register count from 398 to 606 and increases the encryption cycle count to 279 cycles.

The timing results also show an important trade-off. The baseline design achieved a higher Fmax of 72.23 MHz, while the optimized design achieved 57.84 MHz. Both designs still met the 50 MHz timing target, but the baseline design had more timing margin. This suggests that although the optimized design uses fewer logic elements, the added control paths, multiplexing, and shared-resource scheduling can still affect timing performance.

Another engineering lesson is that modular RTL improves readability and verification. The baseline design separates AES operations into modules such as sub_bytes.sv, shift_rows.sv, mix_columns.sv, add_roundkey.sv, and key_expansion.sv. This makes the design easier to understand and debug. However, a highly modular structure is not always the most area-efficient implementation because each module may duplicate resources. The optimized design reduces area by integrating more sequencing inside aes_core.sv, but this makes the control FSM more complex.

Overall, the project shows that the best AES hardware architecture depends on the design target. If low latency and simpler control are more important, the baseline design is preferable. If logic area reduction is more important, the optimized design is more suitable. The final results therefore demonstrate a realistic hardware design trade-off between speed, area, and control complexity.

Table 10.1: Quartus Timing Summary

Metric	Baseline Design	Optimized Design	Engineering Meaning
Logic Elements	4,941	1,459	Optimized design greatly reduces logic area.
Registers	398	606	Optimized design needs more control and temporary storage.
Fmax	72.23 MHz	57.84 MHz	Baseline has higher timing margin.
Cycle Count	11 cycles	279 cycles	Optimized design has much higher latency.

10.2 Design Limitations

Table 10.2 summarises the main limitations of the current AES-128 hardware accelerator. These limitations mainly come from the project scope, the selected standalone interface, the area-focused optimized architecture, and the synthesis setup used for comparison.

Table 10.2: Design Limitations and Possible Improvements

Limitation	Reason	Possible improvement
Encryption only	The project scope focuses only on AES-128 encryption. AES decryption, AES-192, and AES-256 were excluded to keep the design focused and manageable.	Add inverse AES support in a future version.
One 128-bit block processed at a time	The selected start / busy / done interface is simple and suitable for standalone verification, but it only accepts one block per transaction.	Add streaming or memory-mapped block transfer support.
No side-channel protection	The design focuses on functional correctness, synthesis, and area/timing comparison. Side-channel attacks were outside the project scope.	Add hardware-level security countermeasures.
Optimized design has high cycle count	The optimized architecture reuses one S-box and one MixColumns column unit, which reduces area but requires many more cycles.	Use a partially shared architecture to balance area and latency.
Large external data interface	The core uses 128-bit plaintext, 128-bit key, and 128-bit ciphertext ports, which are suitable for simulation but large for small physical wrappers.	Add serial loading, byte-wise loading, or a register-based wrapper.
Limited benchmark comparability	Online AES hardware results often use different FPGA devices, ASIC technologies, clock constraints, synthesis tools, and design goals.	Compare future designs under the same tool, device, and constraints.
Virtual pin compilation setup	Quartus compilation used virtual pins instead of physical FPGA board pin assignments. This is acceptable for synthesis comparison but not a complete board deployment.	Assign physical FPGA pins and repeat board-level timing analysis.

10.3 Future Improvements

Future work should focus on turning the verified standalone AES-128 core into a more practical and balanced hardware accelerator. The first priority is to improve the architecture between the two implemented extremes. The current baseline design gives a useful reference point, while the optimized design greatly reduces logic area but increases the cycle count to 279 cycles. A partially shared design could provide a better compromise, such as using four shared S-boxes instead of one or processing the AES state by column while retaining some byte-level parallelism. This would reduce the latency penalty while still keeping the area lower than the baseline design.

The second priority is to improve system integration. The current 128-bit plaintext, 128-bit key, and 128-bit ciphertext ports are convenient for simulation and direct testbench verification, but they are not ideal for I/O-limited platforms or Tiny Tapeout-style wrappers. A future version should use a byte-wise serial interface, register-based interface, or memory-mapped wrapper so that the AES core can be integrated more easily with a processor, DMA controller, or external control system.

The third priority is to strengthen verification and functionality. AES decryption could be added by implementing the inverse AES transformations and reverse key scheduling. Verification could also be expanded using constrained-random testing, assertion-based checks, and formal properties for FSM sequencing, final-round MixColumns bypass, and output-valid behaviour. This would improve confidence that the optimized design remains correct beyond the selected simulation vectors.

Finally, a more complete cryptographic hardware implementation should consider security hardening. Practical AES hardware can leak information through power consumption, timing behaviour, or electromagnetic emissions. Future work could therefore investigate masking, hiding, randomised execution, or balanced logic. These additions would move the design from a verified standalone AES-128 accelerator toward a more practical, integrated, and security-aware cryptographic hardware block.

11.0 Submission Package Organisation

The final submission package is organised to separate synthesizable RTL, simulation files, backend evidence, reports, and documentation. This structure ensures that the project can be reviewed, simulated, synthesized, and traced clearly.

Table 11.1: Submission Package Reference

Folder	Required Content	Key Files / Notes
rtl/	All synthesizable design files	Contains separate baseline/ and optimized/ RTL folders. Only synthesizable .sv design files are stored here.
tb/	Testbenches and test-vector files	Contains baseline and optimized testbenches, including full AES-128 encryption tests and module-level unit tests. The AES vector file is also stored here for repeatable verification.
quartus/	Quartus project files and settings	Contains the Quartus projects for baseline and optimized designs, including project settings and compilation outputs.
sim/	Simulation scripts, logs, waveform dumps, screenshots	Contains .do scripts, cleaned simulation logs, and .wlf waveform files for verification evidence.
tinytapeout/	Submission files and Tiny Tapeout workflow files	Contains Tiny Tapeout wrapper files, configuration files, GDS logs, layout renders, precheck results, and submission package evidence.
reports/	Exported reports and summaries	Contains resource summaries, timing summaries, hardening summaries, and other generated report evidence.
results/	Generated outputs to preserve	Contains selected netlists, images, layout evidence, and final output artifacts used in the report.
docs/	Final report and citation declaration	Contains the final report, citation declaration, and any supporting documentation required for submission.

This folder structure separates source files from generated outputs. It also makes the project easier to reproduce because the RTL, testbenches, simulation scripts, synthesis reports, and backend artifacts are stored in clearly defined locations.

12.0 Academic Integrity, Citations, and Contribution Statement

12.1 Use of External Sources

External sources were used only as technical references, verification references, and backend workflow templates. The AES-128 algorithm description, round structure, key expansion process, S-box values, ShiftRows operation, MixColumns finite-field equations, AddRoundKey operation, and final-round behaviour were referenced from the official AES standard [1]. Public NIST AES-128 test vectors were used to verify that the implemented RTL produced correct ciphertext outputs for known plaintext and key values [2], [3]. The Tiny Tapeout SKY Verilog template was used as the starting structure for the ASIC-style backend workflow.

All design implementation, RTL integration, module adaptation, wrapper development, simulation testbench construction, debugging, result analysis, and report writing were completed by the group. The external references were not copied as complete design solutions; instead, they were used to guide correct AES functionality, provide credible verification data, and support the Tiny Tapeout submission workflow [4].

12.2 Citation Declaration

Table 12.1: Citation List and Utilization

Source / Material Used	How It Was Used	Citation Number	Original Work Performed By Group
AES standard / algorithm description	Used as the main technical reference for AES-128 encryption, including the 128-bit block size, 128-bit key size, 10-round structure, SubBytes, ShiftRows, MixColumns, AddRoundKey, key expansion, and final-round behaviour.	[1]	Implemented the AES-128 encryption datapath, control FSM, key expansion logic, output handling, and design integration in SystemVerilog.
AES S-box table	Used as the reference substitution mapping for the baseline lookup-table S-box and to verify the optimized fixed-Boolean S-box behaviour.	[5]	Converted the S-box behaviour into RTL form and verified it using both module-level and full-core simulation testbenches.
AES-128 test vectors	Used as known plaintext, key, and ciphertext values for self-checking functional verification.	[2], [3]	Built self-checking testbenches that loaded vectors, waited for the done signal, compared ciphertext outputs, and reported PASS/FAIL results.
Tiny Tapeout SKY Verilog template	Used as the backend repository structure and workflow template for documentation generation, precheck, GDS hardening, and submission packaging.	[4]	Adapted the top-level wrapper, I/O mapping, configuration files, RTL file list, test setup, and Tiny Tapeout repositories for this AES-128 project.

This citation declaration shows which materials were externally referenced and which parts were implemented by the group. The main project contribution is the AES-128 hardware implementation, optimization, verification, synthesis comparison, and Tiny Tapeout backend adaptation.

12.3 Contribution Statement Summary

Table 12.2: Contribution Summary

Member	Main Responsibilities	Shared Responsibilities	Estimated Contribution
Chin Wei Chun	Optimized AES-128 RTL implementation, 10-vector verification setup, Tiny Tapeout backend integration, and backend result analysis.	Debugging, simulation review, report writing, result interpretation, and project demonstration.	50 %
Foong Yao Le	Baseline AES-128 RTL implementation, module-level testbenches, full simulation setup, and verification result collection.	Debugging, simulation review, report writing, result interpretation, and project demonstration.	50 %

Both members contributed equally to the final project outcome. Responsibilities were divided between baseline development, optimized development, verification, backend workflow, debugging, and documentation, while major design decisions and report conclusions were reviewed collaboratively.

13.0 Conclusion

This project successfully implemented, verified, synthesized, and hardened two AES-128 hardware accelerator designs: a baseline implementation and an optimized implementation. The baseline design provided a functionally correct reference architecture with lower encryption latency, while the optimized design focused on reducing hardware area through resource sharing. Both designs implement AES-128 encryption using the same external start/busy/done control concept and were verified using the same reference test-vector methodology.

Functional verification confirmed that both implementations produced correct AES-128 ciphertext outputs. In ModelSim, all 288 selected AES-128 test vectors passed for both the baseline and optimized designs. Additional module-level testbenches were used to verify key AES operations, including SubBytes, ShiftRows, MixColumns, AddRoundKey, key expansion, and AES round-stage behaviour. This layered verification approach provided confidence that both the individual datapath blocks and the complete encryption cores operated correctly.

Quartus synthesis and timing analysis confirmed that both designs were synthesizable and met the 50 MHz target clock. The baseline design achieved lower latency because it uses a more parallel datapath, completing encryption in fewer cycles. However, this came at a higher hardware cost. The optimized design reduced Quartus logic elements from 4,941 to 1,459, demonstrating a significant FPGA area reduction. This reduction was achieved by reusing internal AES resources, including the S-box and MixColumns column logic, although the optimized design required more registers, control states, and clock cycles.

The Tiny Tapeout backend results further confirmed the advantage of the optimized architecture for an area-constrained ASIC-style implementation. The optimized design required only a 4x2 tile allocation, compared with 8x2 tiles for the baseline design. It also reduced backend wire length from 726,069 µm to 259,031 µm and reduced combinational cell count substantially. These results show that the optimized design is more suitable for physical implementation where silicon area, routing demand, and backend feasibility are important constraints.

Overall, the optimized AES-128 accelerator is selected as the preferred final design. Although it has higher latency than the baseline, it maintains correct AES-128 functionality while significantly reducing physical tile usage, routing complexity, and combinational logic count. The project demonstrates that hardware design quality should not be judged by functional correctness alone. A complete engineering evaluation must also consider verification strength, synthesis results, timing closure, backend feasibility, and the trade-off between speed and area. Through this process, the project achieved its objective of developing a correct, synthesizable, and backend-ready AES-128 hardware accelerator with a meaningful area-optimized implementation.

14.0 AI Use Declaration

AI tools were used in this project as an assistive support tool for report preparation, technical clarification, and writing refinement. The use of AI did not replace the engineering work, design implementation, simulation execution, synthesis, backend workflow, or final technical judgement completed by the project group.

The project group used ChatGPT by OpenAI in the following ways:

1. **Idea generation, structure suggestions, and preparatory support**
   ChatGPT was used to support foundational understanding of the AES-128 hardware accelerator project, including AES-128 documentation, Tiny Tapeout workflow references, Quartus and ModelSim-related workflow guidance, and suitable ways to organise the report. It was also used to suggest section structures, table layouts, and presentation formats for explaining the AES-128 design, verification strategy, synthesis results, backend workflow, and engineering trade-off analysis. ChatGPT is also used to generate repetitive codes used in verification process where test case information was prepared by the group.

2. **Text refinement, rewriting, and paraphrasing support**
   ChatGPT was used to refine grammar, sentence structure, clarity, academic tone, and transitions between report sections. This included improving the wording of technical explanations, making paragraphs more concise, and presenting the engineering discussion in a clearer and more professional manner. All AI-assisted wording was reviewed, edited, and adapted by the group before inclusion in the final report to ensure that it accurately reflected the actual project work and results.

3. **Formatting, summarisation, and presentation support**
   ChatGPT was used to assist with formatting report sections, preparing summary tables, improving the executive summary and conclusion, and suggesting how the report could better align with the project requirements and marking expectations.

The outputs from ChatGPT were not submitted without review. The group checked, edited, and adapted all AI-assisted content to ensure that the final report accurately represented the implemented AES-128 RTL designs, ModelSim verification results, Quartus synthesis results, Tiny Tapeout backend evidence, and engineering analysis. The RTL implementation, simulation execution, Quartus compilation, Tiny Tapeout workflow, screenshots, logs, measured results, and final technical conclusions were completed and verified by the project group.

AI was not used to fabricate results, generate false evidence, replace technical implementation work, or substitute the group's engineering judgement.

15.0 References

[1] N. I. of S. and Technology, "Advanced Encryption Standard (AES)," *csrc.nist.gov*, May 09, 2023. Available: https://csrc.nist.gov/pubs/fips/197/final

[2] M. Dworkin, "Recommendation for Block Cipher Modes of Operation Methods and Techniques," 2001. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf

[3] L. Bassham, "The Advanced Encryption Standard Algorithm Validation Suite (AESAVS)," 2002. Available: https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Algorithm-Validation-Program/documents/aes/AESAVS.pdf

[4] TinyTapeout, "GitHub - TinyTapeout/ttsky-verilog-template: Submission template for Tiny Tapeout SKY130 (ChipFoundry) shuttles - Verilog HDL Projects," *GitHub*, 2025. Available: https://github.com/TinyTapeout/ttsky-verilog-template.

[5] usnistgov, "GitHub - usnistgov/Circuits: Circuits for functions of interest to cryptography," *GitHub*, 2025. Available: https://github.com/usnistgov/Circuits.

Appendix A: RTL Source File Summary

This appendix summarises the main synthesizable RTL source files used in the baseline and optimized AES-128 hardware accelerator implementations. The purpose of this appendix is to make the submitted RTL structure easier to inspect and to show how each source file contributes to the AES-128 encryption datapath, control logic, key expansion, and output handling.

Table A.1: Baseline RTL Source File Summary

File	Purpose	Important design notes
aes128_baseline.sv	Top-level wrapper for the baseline AES-128 design.	Provides the external clock, reset, start, plaintext, key, ciphertext, busy, and done interface. It instantiates the baseline AES core and exposes the simple start/busy/done control interface used for simulation and synthesis.
aes_core.sv	Main baseline AES controller and state register module.	Controls the initial AddRoundKey step, AES round sequencing, round counter, round-key register, busy signal, done signal, and ciphertext output. The baseline core is arranged so that one AES round is completed per round cycle.
aes_round.sv	Complete AES round datapath for the baseline design.	Combines SubBytes, ShiftRows, MixColumns, and AddRoundKey into one round datapath. The final_round control signal bypasses MixColumns during round 10, matching the AES-128 encryption structure.
sub_bytes.sv	Parallel SubBytes transformation for the 128-bit AES state.	Instantiates 16 S-box operations so that all 16 bytes of the AES state are substituted in parallel. This supports the lower cycle count of the baseline architecture but increases combinational logic usage.
sbox.sv	AES S-box substitution block.	Implements the 8-bit input to 8-bit output AES substitution function using a lookup-table style implementation. It is used by the SubBytes operation and by the key expansion SubWord operation.
shift_rows.sv	AES ShiftRows byte permutation block.	Rearranges the 128-bit AES state according to the AES row-shifting rule. This block is implemented as combinational byte rewiring and does not require additional sequential control.
mix_columns.sv	Full-state MixColumns transformation for the baseline design.	Processes all four 32-bit AES columns in parallel using finite-field XOR and xtime logic. The output is used during rounds 1 to 9 and bypassed during the final round.
add_roundkey.sv	AddRoundKey transformation block.	Performs a 128-bit XOR between the current AES state and the selected 128-bit round key. This operation is purely combinational and is used in the initial step and every AES round.
key_expansion.sv	Baseline AES-128 round-key generation module.	Generates the next 128-bit round key from the current round key using RotWord, SubWord, Rcon, and XOR chaining. The combinational implementation allows the next round key to be produced alongside the round datapath.

Table A.2: Optimized RTL Source File Summary

File	Purpose	Important design notes
aes128_optimized.sv	Top-level wrapper for the optimized AES-128 design.	Provides the same external clock, reset, start, plaintext, key, ciphertext, busy, and done interface as the baseline design. This allows both designs to be verified using the same testbench structure.
aes_core.sv	Main optimized AES controller, FSM, and shared-resource datapath.	Contains the optimized multi-cycle FSM, state register, round counter, byte counter, column counter, shared S-box scheduling, key expansion sequencing, MixColumns sequencing, AddRoundKey operation, busy control, done control, and ciphertext output handling.
sbox.sv	Shared AES S-box substitution block.	Implements the AES substitution function using fixed Boolean logic. In the optimized design, one S-box is reused for both key expansion SubWord and AES SubBytes, reducing duplicated S-box hardware at the cost of additional clock cycles.
shift_rows.sv	AES ShiftRows byte permutation block.	Performs the AES ShiftRows operation as combinational byte rewiring. This block is not sequentially shared because its logic cost is relatively small compared with the S-box and MixColumns logic.
mix_columns.sv	Shared one-column MixColumns block.	Implements a single 32-bit MixColumns column operation. The optimized FSM reuses this one-column unit across the four AES state columns using a column counter, reducing duplicated MixColumns hardware.

Table A.3: Tiny Tapeout Wrapper RTL Summary

File / module	Purpose	Important design notes
tt_um_aes128_baseline	Tiny Tapeout top module for the baseline AES-128 design.	Converts the original wide AES interface into a Tiny Tapeout-compatible 8-bit byte-loading interface. It loads the 128-bit key and plaintext byte-by-byte, starts encryption, and reads the 128-bit ciphertext byte-by-byte.
tt_um_aes128_optimized	Tiny Tapeout top module for the optimized AES-128 design.	Provides the same Tiny Tapeout byte-serial wrapper concept for the optimized AES core. This allows the optimized design to satisfy the limited Tiny Tapeout I/O interface while preserving AES-128 encryption behaviour.
Byte-loading control logic	Loads key and plaintext bytes into internal 128-bit registers.	Uses wrapper control inputs such as load_key and load_plaintext to store 16 input bytes into the internal key and plaintext registers before encryption begins.
Ciphertext readout logic	Outputs ciphertext bytes through the 8-bit Tiny Tapeout output port.	Uses the read_ciphertext_next control signal to output the 128-bit ciphertext one byte at a time after done is asserted. This avoids exposing a full 128-bit ciphertext bus at the chip boundary.
Status output logic	Provides wrapper-level busy, done, and ready signals.	Allows the external Tiny Tapeout testbench or interface logic to identify whether the AES core is idle, processing, or finished.

Overall, the baseline RTL is more modular and parallel, with separate source files for the main AES transformations and a one-round-per-cycle datapath. This improves readability and reduces cycle count but increases duplicated combinational hardware. The optimized RTL integrates more control into the main AES core and reuses selected datapath resources, especially the S-box and MixColumns column unit. This reduces logic area and Tiny Tapeout backend footprint, but increases FSM complexity, register usage, and encryption cycle count.

Appendix B: Additional Screenshots and Logs

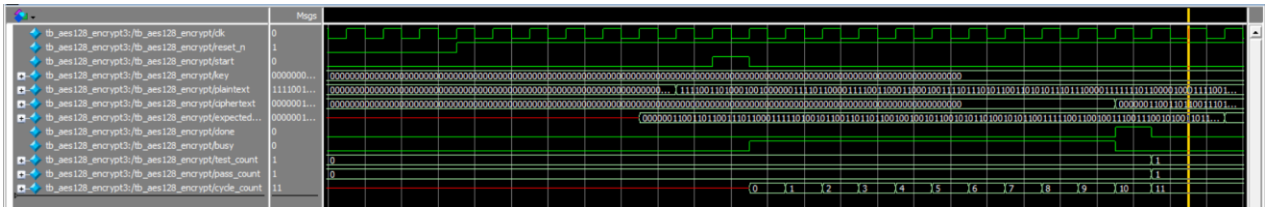

Figure B.1: Baseline AES-128 Single Test Vector Verification Waveform

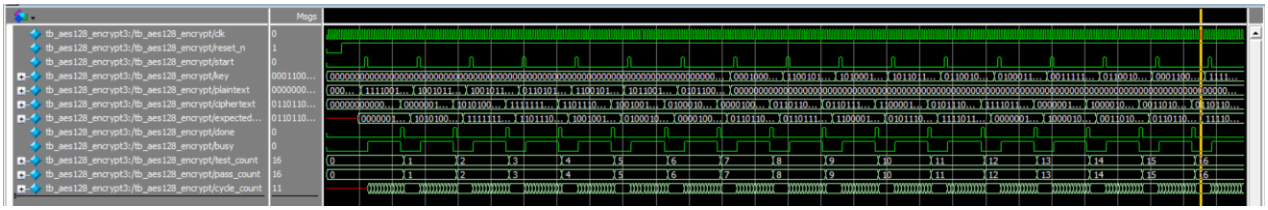

Figure B.2: Baseline AES-128 Multiple Test Vector Verification Waveform

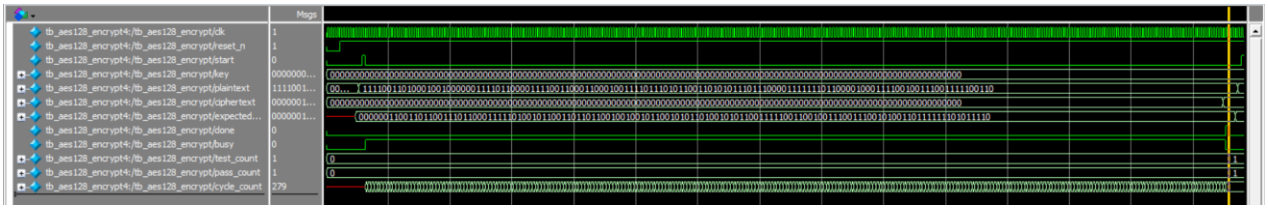

Figure B.3: Optimized AES-128 Single Test Vector Verification Waveform

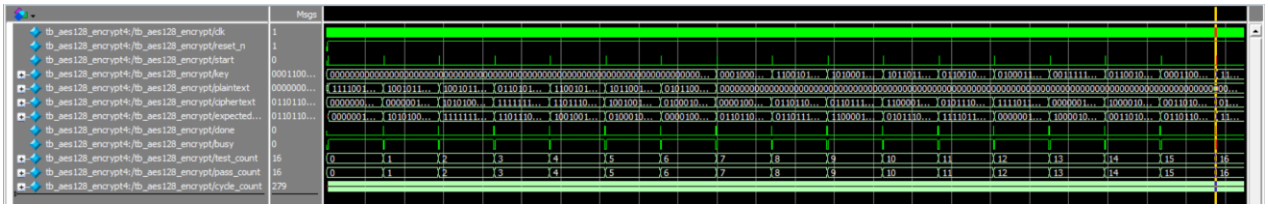

Figure B.4: Optimized AES-128 Multiple Test Vector Verification Waveform

```
# ------------------------------------------------
# TEST 10: fips197_round2_addroundkey
# Operation : AddRoundKey
# STATE_IN  = 584dcaf11b4b5aacdbe7caa81b6bb0e5
# ROUND_KEY = f2c295f27a96b9435935807a7359f67f
# EXPECTED  = aa8f5f0361dde3ef82d24ad26832469a
# GOT       = aa8f5f0361dde3ef82d24ad26832469a
# STATUS    = PASS
# ------------------------------------------------
# TEST 11: fips197_round3_addroundkey
# Operation : AddRoundKey
# STATE_IN  = 75ec0993200b633353c0cf7cbb25d0dc
# ROUND_KEY = 3d80477d4716fe3e1e237e446d7a883b
# EXPECTED  = 486c4eee671d9d0d4de3b138d65f58e7
# GOT       = 486c4eee671d9d0d4de3b138d65f58e7
# STATUS    = PASS
# ------------------------------------------------
# TEST 12: fips197_final_addroundkey
# Operation : AddRoundKey
# STATE_IN  = e9317db5cb322c723d2e895faf090794
# ROUND_KEY = d014f9a8c9ee2589e13f0cc8b6630ca6
# EXPECTED  = 3925841d02dc09fbdc118597196a0b32
# GOT       = 3925841d02dc09fbdc118597196a0b32
# STATUS    = PASS
# ================================================
# AES AddRoundKey Unit Test Summary
# Total tests : 12
# Passed      : 12
# Failed      : 0
# ================================================
# RESULT: ALL ADDROUNDKEY TESTS PASSED
```

Figure B.5: Baseline AES-128 AddRoundKey Test Passed

```
# ------------------------------------------------
# TEST 11: Round 8 full output
# EXPECTED = ea835cf00445332d655d98ad8596b0c5
# GOT      = ea835cf00445332d655d98ad8596b0c5
# STATUS   = PASS
# Round input state = ea835cf00445332d655d98ad8596b0c5
# Round key         = ac7766f319fadc2128d12941575c006e
# Final round flag  = 0
# ------------------------------------------------
# TEST 12: Round 9 full output
# EXPECTED = eb40f21e592e38848ba113e71bc342d2
# GOT      = eb40f21e592e38848ba113e71bc342d2
# STATUS   = PASS
# Round input state = eb40f21e592e38848ba113e71bc342d2
# Round key         = d014f9a8c9ee2589e13f0cc8b6630ca6
# Final round flag  = 1
# ------------------------------------------------
# TEST 13: Round 10 final-round output
# EXPECTED = 3925841d02dc09fbdc118597196a0b32
# GOT      = 3925841d02dc09fbdc118597196a0b32
# STATUS   = PASS
# ================================================
# AES Round Stage Summary
# Total tests : 13
# Passed      : 13
# Failed      : 0
# ================================================
# RESULT: AES ROUND STAGE TEST PASSED
```

Figure B.6: Baseline AES-128 Round Stage Test Passed

```
# ------------------------------------------------
# TEST 8: K7_to_K8
# Operation = AES-128 one-round key expansion
# RCON      = 80
# KEY_IN    = 4e54f70e5f5fc9f384a64fb24ea6dc4f
# EXPECTED  = ead27321b58dbad2312bf5607f8d292f
# GOT       = ead27321b58dbad2312bf5607f8d292f
# STATUS    = PASS
# ------------------------------------------------
# TEST 9: K8_to_K9
# Operation = AES-128 one-round key expansion
# RCON      = 1b
# KEY_IN    = ead27321b58dbad2312bf5607f8d292f
# EXPECTED  = ac7766f319fadc2128d12941575c006e
# GOT       = ac7766f319fadc2128d12941575c006e
# STATUS    = PASS
# ------------------------------------------------
# TEST 10: K9_to_K10
# Operation = AES-128 one-round key expansion
# RCON      = 36
# KEY_IN    = ac7766f319fadc2128d12941575c006e
# EXPECTED  = d014f9a8c9ee2589e13f0cc8b6630ca6
# GOT       = d014f9a8c9ee2589e13f0cc8b6630ca6
# STATUS    = PASS
# ================================================
# KeyExpansion Summary
# Total tests : 10
# Passed      : 10
# Failed      : 0
# ================================================
# RESULT: KEYEXPANSION UNIT TEST PASSED
```

Figure B.7: Baseline AES-128 KeyExpansion Test Passed

```
# ------------------------------------------------
# TEST 14: mixed_pattern_1
# Operation = full 128-bit MixColumns
# INPUT     = 0123456789abcdeffedcba9876543210
# EXPECTED  = 45ef01abcd678923ba10fe54329876dc
# GOT       = 45ef01abcd678923ba10fe54329876dc
# STATUS    = PASS
# ------------------------------------------------
# TEST 15: nist_plaintext_block_1
# Operation = full 128-bit MixColumns
# INPUT     = 6bc1bee22e409f96e93d7e117393172a
# EXPECTED  = d2c9f01d9582ea9ae1001b41755db045
# GOT       = d2c9f01d9582ea9ae1001b41755db045
# STATUS    = PASS
# ------------------------------------------------
# TEST 16: nist_plaintext_block_2
# Operation = full 128-bit MixColumns
# INPUT     = ae2d8a571e03ac9c9eb76fac45af8e51
# EXPECTED  = ed2675e0096be1ae26f61822bfd81e4c
# GOT       = ed2675e0096be1ae26f61822bfd81e4c
# STATUS    = PASS
# ================================================
# Full MixColumns Summary
# Total tests : 16
# Passed      : 16
# Failed      : 0
# ================================================
# RESULT: FULL MIXCOLUMNS UNIT TEST PASSED
```

Figure B.8: Baseline AES-128 Full Mix Column Test Passed

```
# ------------------------------------------------
# TEST 253: SBOX(fc)
# INPUT    = fc
# EXPECTED = b0
# GOT      = b0
# STATUS   = PASS
# ------------------------------------------------
# TEST 254: SBOX(fd)
# INPUT    = fd
# EXPECTED = 54
# GOT      = 54
# STATUS   = PASS
# ------------------------------------------------
# TEST 255: SBOX(fe)
# INPUT    = fe
# EXPECTED = bb
# GOT      = bb
# STATUS   = PASS
# ------------------------------------------------
# TEST 256: SBOX(ff)
# INPUT    = ff
# EXPECTED = 16
# GOT      = 16
# STATUS   = PASS
# ================================================
# S-box Summary
# Total tests : 256
# Passed      : 256
# Failed      : 0
# ================================================
# RESULT: SBOX UNIT TEST PASSED
```

Figure B.9: Baseline AES-128 SBox Test Passed

```
# EXPECTED  = f783403f27433df09bb531ff54aba9d3
# GOT       = f783403f27433df09bb531ff54aba9d3
# STATUS    = PASS
# ------------------------------------------------
# TEST 13: fips197_round8_subbytes_to_shiftrows
# Operation = ShiftRows
# INPUT     = be832cc8d43b86c00ae1d44dda64f2fe
# EXPECTED  = be3bd4fed4e1f2c80a642cc0da83864d
# GOT       = be3bd4fed4e1f2c80a642cc0da83864d
# STATUS    = PASS
# ------------------------------------------------
# TEST 14: fips197_round9_subbytes_to_shiftrows
# Operation = ShiftRows
# INPUT     = 87ec4a8cf26ec3d84d4c46959790e7a6
# EXPECTED  = 876e46a6f24ce78c4d904ad897ecc395
# GOT       = 876e46a6f24ce78c4d904ad897ecc395
# STATUS    = PASS
# ------------------------------------------------
# TEST 15: fips197_round10_subbytes_to_shiftrows
# Operation = ShiftRows
# INPUT     = e9098972cb31075f3d327d94af2e2cb5
# EXPECTED  = e9317db5cb322c723d2e895faf090794
# GOT       = e9317db5cb322c723d2e895faf090794
# STATUS    = PASS
# ================================================
# ShiftRows Summary
# Total tests : 15
# Passed      : 15
# Failed      : 0
# ================================================
# RESULT: SHIFTROWS UNIT TEST PASSED
```

Figure B.10: Baseline AES-128 ShiftRows Test Passed

```
# EXPECTED  = f7ab31f02783a9ff9b4340d354b53d3f
# GOT       = f7ab31f02783a9ff9b4340d354b53d3f
# STATUS    = PASS
# ------------------------------------------------
# TEST 14: fips197_round8_start_to_subbytes
# Operation = SubBytes
# STATE_IN  = 5a4142b11949dc1fa3e019657a8c040c
# EXPECTED  = be832cc8d43b86c00ae1d44dda64f2fe
# GOT       = be832cc8d43b86c00ae1d44dda64f2fe
# STATUS    = PASS
# ------------------------------------------------
# TEST 15: fips197_round9_start_to_subbytes
# Operation = SubBytes
# STATE_IN  = ea835cf00445332d655d98ad8596b0c5
# EXPECTED  = 87ec4a8cf26ec3d84d4c46959790e7a6
# GOT       = 87ec4a8cf26ec3d84d4c46959790e7a6
# STATUS    = PASS
# ------------------------------------------------
# TEST 16: fips197_round10_start_to_subbytes
# Operation = SubBytes
# STATE_IN  = eb40f21e592e38848ba113e71bc342d2
# EXPECTED  = e9098972cb31075f3d327d94af2e2cb5
# GOT       = e9098972cb31075f3d327d94af2e2cb5
# STATUS    = PASS
# ================================================
# AES SubBytes Unit Test Summary
# Total tests : 16
# Passed      : 16
# Failed      : 0
# ================================================
# RESULT: ALL SUBBYTES TESTS PASSED
```

Figure B.11: Baseline AES-128 SubBytes Test Passed

```
# ------------------------------------------------
# TEST 20: mixed_pattern_1
# Operation = one-column MixColumns
# INPUT     = 01234567
# EXPECTED  = 45ef01ab
# GOT       = 45ef01ab
# STATUS    = PASS
# ------------------------------------------------
# TEST 21: mixed_pattern_2
# Operation = one-column MixColumns
# INPUT     = 89abcdef
# EXPECTED  = cd678923
# GOT       = cd678923
# STATUS    = PASS
# ------------------------------------------------
# TEST 22: mixed_pattern_3
# Operation = one-column MixColumns
# INPUT     = fedcba98
# EXPECTED  = ba10fe54
# GOT       = ba10fe54
# STATUS    = PASS
# ------------------------------------------------
# TEST 23: mixed_pattern_4
# Operation = one-column MixColumns
# INPUT     = 76543210
# EXPECTED  = 329876dc
# GOT       = 329876dc
# STATUS    = PASS
# ================================================
# MixColumns One-Column Summary
# Total tests : 23
# Passed      : 23
# Failed      : 0
# ================================================
# RESULT: MIXCOLUMNS ONE-COLUMN UNIT TEST PASSED
```

Figure B.12: Baseline AES-128 One Column Test Passed